Contents

1. Introduction 8
1.1 Background and Motivation 9
1.2 Problem Statement 11
1.3 Objectives 12
1.3.1 Primary Objective 13
1.3.2 Specific Objectives 13
1.4 Scope and Limitations 15
Scope 15
Limitations 17
2. Literature Survey / Existing System 19
2.1 Review of Existing Systems 20
2.1.1 Conventional Battery Management Systems 20
2.1.2 IoT-Based Battery Monitoring Systems 22
2.1.3 Cloud-Connected Battery Monitoring 23
2.1.4 Machine-Learning-Based Anomaly Detection 25
2.2 Gaps Identified and Proposed Solution 27
2.2.1 Gaps Identified 28
2.2.2 Proposed Solution: Secure-BMS 31
3. Software Process Model and Project Planning 34
3.1 SDLC Model Chosen and Justification 34
3.1.1 Selected Software Development Life Cycle Model 34
3.1.2 Application of Agile in Secure-BMS 36
3.1.3 Justification for the Selected Model 38
3.2 Project Schedule (Gantt Chart / Sprint Plan) 39
3.2.1 Development Schedule 40
3.2.2 Gantt Chart 42
3.3 Risk Analysis 42
3.3.1 Risk Identification and Mitigation 43
3.3.2 Risk Assessment 45
3.4 Cost and Effort Estimation (Optional) 46
3.4.1 Hardware Cost Estimation 47
3.4.2 Software Cost Estimation 47
3.4.3 Effort Estimation 48
3.4.4 Cost and Effort Limitations 49
4. Requirements Analysis 49
4.1 Functional Requirements 49

4.2 Non-Functional Requirements 51
4.3 Software Requirements Specification (SRS) Summary 53
4.3.1 Purpose 53
4.3.2 System Overview 53
4.3.3 Inputs 53
4.3.4 Processing 54
4.3.5 Outputs 54
4.3.6 Assumptions and Constraints 55
4.4 Hardware and Software Requirements 56
4.4.1 Hardware Requirements 56
4.4.2 Software Requirements 56
Chapter Summary 57
5. System Design 58
5.1 System Architecture 58
5.1.1 Overview 58
5.1.2 Architectural Components 58
5.1.3 System Data Flow 59
5.2 Use Case Diagram 60
Actors 60
Principal Use Cases 61
5.3 Class Diagram 61
Main Components 62
5.4 Sequence Diagram / Activity Diagram 64
5.4.1 Sequence Diagram 65
5.4.2 Activity Diagram 66
5.5 ER Diagram / Database Design 66
5.5.1 Database Overview 66
5.5.2 Telemetry Entity 67
5.5.3 Anomaly History Entity 67
5.5.4 Relationship Between Entities 68
5.6 UI Design / Wireframes 68
Chapter Summary 69
6. Implementation 70
6.1 Introduction 70
6.2 Hardware Implementation 70
6.2.1 ESP32 Microcontroller Setup 70
6.2.2 Temperature Sensing Circuit 71

6.2.3 Comparator and Local Alert Circuit 71
6.2.4 Load-Switching Hardware 71
6.2.5 Battery and Supporting Components 71
6.3 Embedded Firmware Implementation 72
6.3.1 Firmware Development Environment 72
6.3.2 Sensor Acquisition and Filtering 72
6.3.3 Event-State Management 72
6.3.4 Wi-Fi Connectivity and Recovery 73
6.4 Secure Telemetry Implementation 73
6.4.1 AES-128-GCM Encryption 73
6.4.2 Nonce Generation and Message Counter 73
6.4.3 MQTT Telemetry Publication 74
6.5 Backend Implementation 74
6.5.1 Backend Architecture 74
6.5.2 Message Decryption and Validation 74
6.5.3 MQTT Message Handling 74
6.6 Replay-Protection Implementation 75
6.6.1 Replay-Protection Mechanism 75
6.6.2 Database-Backed Replay State 75
6.6.3 Replay-Protection Testing 75
6.7 Database Implementation 76
6.7.1 PostgreSQL Integration 76
6.7.2 Transaction Management 76
6.7.3 Data Integrity 76
6.8 Dashboard Implementation 76
6.8.1 Streamlit Dashboard 76
6.8.2 Monitoring Information 77
6.8.3 Integration Limitations 77
6.9 Anomaly-Detection Component 77
6.10 Software Integration 77
6.11 Implementation Status 78
6.12 Chapter Summary 79
7. Software Tools Used (Demo Screenshots) 79
7.1 Arduino IDE 79
7.2 Visual Studio Code 80
7.3 Git and GitHub 81
7.4 Mosquitto MQTT Broker 82

7.5 PostgreSQL 83
7.6 Streamlit 84
7.7 Jira 85
Chapter Summary 85
8. Testing 86
8.1 Testing Strategy 86
8.2 Test Cases 86
8.3 Hardware Testing Results 88
8.3.1 ESP32 and ADC Testing 88
8.3.2 Temperature-Sensing Testing 88
8.3.3 Comparator and Buzzer Testing 88
8.3.4 Load-Control Interface 89
8.3.5 Hardware Testing Summary 89
8.4 Defects and Limitations Identified 89
8.4.1 Temperature Measurement Accuracy 89
8.4.2 Current Measurement Validation 89
8.4.3 End-to-End Security Validation 89
8.4.4 Automated Load Disconnection 90
8.4.5 Anomaly-Detection Evaluation 90
8.4.6 System Integration and Safety 90
8.4.7 Summary of Limitations 90
8.5 Testing Summary 91
9. Results and Discussion 91
9.1 Prototype Implementation Results 91
9.2 Results Against Project Objectives 92
9.3 Discussion 94
Temperature Monitoring and Local Alerting 94
Secure Telemetry and Replay Protection 94
Database and Dashboard Integration 94
Anomaly Detection 94
9.4 Limitations 95
9.5 Overall Evaluation 96
10. Conclusion and Future Enhancements 96
10.1 Conclusion 96
10.2 Future Enhancements 97
10.2.1 Improved Temperature Measurement 97
10.2.2 Validated Voltage and Current Monitoring 97

10.2.3 Automated Load Management 98
10.2.4 Enhanced Telemetry Security 98
10.2.5 Improved Anomaly Detection 98
10.2.6 Dashboard Improvements 98
10.2.7 Comprehensive Testing and Reliability Evaluation 99
10.3 Future Scope 99
10.3.1 Multi-Cell Battery Monitoring 99
10.3.2 Cloud-Based Monitoring 99
10.3.3 Predictive Analytics and Battery Health Estimation 100
10.3.4 Digital Twin Integration 100
10.3.5 Advanced Fault Detection and Recovery 100
10.3.6 Scalable Monitoring and Research Applications 100
11. References 101
Appendix A: GitHub Repository and Demonstration Video 101
A.1 GitHub Repository 101
A.2 Demonstration Video 102

1. Introduction

Battery-powered systems are increasingly used in portable electronics, electric mobility, energy storage, and other applications that require reliable electrical power. As these

applications become more dependent on rechargeable batteries, monitoring battery operating conditions and identifying potentially unsafe conditions become important engineering requirements. Lithium-ion batteries offer useful characteristics such as high energy density and rechargeable operation; however, their performance and safe operation depend on factors including temperature, electrical loading, cell condition, and the effectiveness of the associated monitoring and protection mechanisms.

A Battery Management System (BMS) is responsible for monitoring relevant battery parameters and supporting safe battery operation. Depending on its design, a BMS may measure voltage, current, and temperature, identify abnormal conditions, and initiate protective actions. Conventional monitoring and protection functions can be extended using embedded controllers, communication networks, databases, and analytical software. These extensions make it possible to observe battery behaviour beyond the immediate hardware interface and to maintain a history of recorded measurements.

The **Secure-BMS (Secure Battery Management System)** project focuses on developing a cyber-physical prototype for monitoring the temperature and operating state of a physical lithium-ion battery cell during a controlled discharge experiment. The prototype combines physical sensing, embedded processing, local alert logic, wireless telemetry, secure message handling, persistent data storage, and a software dashboard. Its design connects the physical behaviour of the battery to software components that process, communicate, store, and display the resulting measurements.

At the hardware level, an ESP32 microcontroller processes measurements obtained through an ADS1115 analogue-to-digital converter and a negative temperature coefficient (NTC) thermistor circuit. The thermistor used in the prototype is approximately 100 kΩ at its nominal reference temperature, with a B3950 temperature coefficient, and is connected in a voltage-divider arrangement using a 10 kΩ resistor. A comparator based on the LM339 provides a separate hardware signal for temperature-threshold detection. An active-low buzzer is connected to GPIO25, while an IRLZ44N MOSFET gate is connected to GPIO27 for the prototype's load-control interface. The comparator output is connected to GPIO26.

The embedded firmware performs temperature measurement processing, checks for sensor-related faults, evaluates operating states, and manages the associated alert outputs. It also prepares telemetry containing operating-state information and selected monitoring statistics. The reported states include `NORMAL`, `TEMPERATURE_ALERT`, and `SENSOR_FAULT`. A median-filtering method is used to reduce the influence of isolated fluctuations in analogue readings.

At the communication and software levels, the project uses Wi-Fi and MQTT to transmit telemetry. The current implementation includes AES-128-GCM authenticated encryption for telemetry payloads, with the MQTT topic used as additional authenticated data. The receiving backend validates and decrypts encrypted messages before processing them. PostgreSQL is used for persistent telemetry storage, and the backend uses FastAPI and Python-based MQTT processing. A Streamlit dashboard provides a software interface for viewing project data. Replay-protection logic has also been implemented in the backend and tested in isolated unit and database-level test scenarios. However, an end-to-end replay attack test through the live MQTT callback has not yet been verified and is therefore not treated as a completed integration-test result.

Secure-BMS is developed as a prototype and educational engineering project rather than a certified commercial battery protection product. Its purpose is to demonstrate how embedded sensing, local alert mechanisms, secure communication, data persistence, and software monitoring can be integrated into a cyber-physical system. The project also provides a foundation for future work involving more comprehensive electrical measurements, machine-learning-based anomaly detection, improved validation, and more advanced battery safety functions.

1.1 Background and Motivation

Lithium-ion batteries are widely used because they can store a relatively large amount of energy for their size and can be recharged for repeated use. Despite these advantages, a battery must operate within appropriate electrical and thermal conditions. Excessive temperature, abnormal voltage, unsuitable loading, sensor malfunction, and failures in monitoring or communication can affect the reliability of a battery-powered system.

A monitoring system is useful only when its measurements are sufficiently reliable and abnormal conditions can be identified in a timely manner. For example, a temperature sensor may produce fluctuating readings because of analogue measurement noise. If every small fluctuation is treated as a meaningful temperature change, the resulting monitoring output may become unstable. Similarly, a disconnected or malfunctioning sensor may produce readings that should not be interpreted as valid physical measurements. These issues motivate the use of signal-processing techniques and explicit fault-handling logic in embedded battery monitoring.

Another consideration is the separation between **local protection** and **remote monitoring**. A cloud-connected dashboard can help an operator observe measurements and review stored data, but network communication may be delayed or unavailable. A safety-related response should therefore not depend entirely on the availability of a remote dashboard. Secure-BMS addresses this design consideration by incorporating local embedded processing and alert-related hardware alongside its telemetry and software components. The prototype includes a comparator-based signal, buzzer output, and MOSFET control interface, while the firmware evaluates sensor readings and operating states locally.

Connectivity also introduces security concerns. Battery telemetry transmitted over a network may contain operational information that should not be accepted without appropriate validation. An attacker or faulty client could attempt to alter a message, submit malformed data, or resend an earlier valid message. Encryption alone does not automatically prevent every such problem. Secure-BMS therefore incorporates authenticated encryption and backend validation, with additional replay-state tracking intended to reject duplicate, stale, or previously observed message sequences.

The project is motivated by the opportunity to study these concerns together in a practical cyber-physical system. Rather than treating the battery sensor, embedded controller, communication broker, database, and dashboard as independent components, Secure-BMS brings them into one prototype architecture. This makes it possible to examine the flow of information from physical measurement to software presentation and to investigate how measurement reliability, communication security, and persistent storage affect the overall design.

The principal motivations for the project are summarised below.

Motivation	Engineering significance	Relevance to Secure-BMS
Temperature monitoring	Temperature is an important operating parameter for lithium-ion cells.	An NTC thermistor circuit is used to obtain temperature-related measurements.
Reliable sensor processing	Analogue readings can fluctuate or become invalid.	The firmware uses median filtering and sensor-fault checks.
Local alert handling	A remote interface may be unavailable during a network failure.	Local comparator and buzzer-related hardware are included in the prototype.
Controlled load interface	Battery discharge experiments require a defined load arrangement.	An IRLZ44N MOSFET provides the prototype's load-control interface; automated load operation must be distinguished from interface availability.
Connected monitoring	Networked telemetry enables measurements to be processed and stored outside the embedded device.	Wi-Fi and MQTT are used to transmit telemetry.
Communication security	Telemetry should be authenticated and validated before being trusted.	AES-128-GCM encryption and backend payload validation are implemented.
Replay resistance	Previously valid messages should not automatically be accepted as new measurements.	Backend replay-state logic has been implemented and tested in isolated scenarios.
Persistent records	Stored measurements support later inspection and analysis.	PostgreSQL is used to store accepted telemetry.
Software visibility	Operators need a convenient way to inspect monitoring information.	A Streamlit dashboard is included in the software architecture.

The project also has an educational motivation. It provides practical experience with embedded programming, analogue sensing, microcontroller interfaces, MQTT messaging, authenticated encryption, backend development, relational database operations, and dashboard development. Integrating these technologies provides a concrete example of how hardware and software engineering decisions interact in a cyber-physical system.

1.2 Problem Statement

A battery monitoring prototype must acquire relevant physical measurements, process those measurements reliably, identify abnormal operating conditions, communicate monitoring information, and retain accepted records for inspection. When communication and remote software components are introduced, the system must also address the integrity and authenticity of transmitted information and avoid treating replayed telemetry as new data.

A design that relies only on periodic sensor readings may not adequately distinguish normal operation from a temperature alert or a sensor fault. A design that depends entirely on remote

monitoring may also become less useful when the network or backend is unavailable. Furthermore, transmitting telemetry without authentication and validation can expose the receiving application to altered, malformed, or repeated messages.

The problem addressed by Secure-BMS is therefore:

To design and implement a prototype cyber-physical battery monitoring system that combines temperature sensing, embedded operating-state evaluation, local alert-related hardware, secure MQTT telemetry, backend validation, replay-state tracking, persistent storage, and dashboard-based monitoring, while clearly identifying the limitations of the implemented prototype.

The problem is considered across five connected areas:

1. **Physical measurement:** obtaining temperature-related readings from a thermistor circuit through an analogue-to-digital converter.
2. **Embedded processing:** reducing measurement fluctuations, checking for sensor faults, evaluating operating states, and managing alert-related outputs.
3. **Secure communication:** transmitting telemetry through MQTT and authenticating encrypted payloads at the receiving backend.
4. **Data management:** validating incoming telemetry, applying replay checks, and saving accepted records to PostgreSQL.
5. **Software monitoring:** presenting the collected information through a dashboard for inspection.

These areas must work together to provide a useful monitoring workflow. The quality of the overall system depends not only on the individual components but also on the correctness of the interfaces between them. For example, an encrypted message must be authenticated and decoded before its fields are processed, and replay-state updates must be coordinated with telemetry insertion so that rejected messages do not become accepted monitoring records.

The project does not claim to solve every battery-management problem. The current prototype concentrates on temperature monitoring, selected fault and alert states, secure telemetry processing, and data storage. Comprehensive state-of-charge estimation, cell balancing, extensive electrical characterisation, certified safety protection, and production-grade validation are outside the demonstrated implementation unless separately developed and tested.

The problem can be expressed through the following system requirements.

Problem area	Required system behaviour	Implementation status
Temperature acquisition	Obtain temperature-related readings from the thermistor circuit.	Implemented and exercised on the prototype.
Measurement stability	Reduce the effect of isolated analogue reading fluctuations.	Median filtering implemented in firmware.
Fault and state evaluation	Identify defined sensor-fault and temperature-alert	Implemented in the embedded firmware.

	conditions.	
Local alert interface	Provide comparator and buzzer-related hardware signals.	Hardware components and signal paths have been tested during prototype development; this is not a certification of protective performance.
Load-control interface	Provide an ESP32-controlled MOSFET gate interface.	Hardware interface implemented; automated load switching is not claimed as an active operating feature while `LOAD_ENABLE = false`.
Telemetry communication	Send monitoring data over Wi-Fi and MQTT.	Implemented in the prototype.
Payload authenticity	Reject encrypted payloads that fail AES-GCM authentication.	Implemented in the backend message-processing path.
Replay detection	Reject repeated or stale telemetry according to stored nonce/session state.	Replay-state logic implemented and isolated tests passed; full live MQTT-path replay testing remains pending.
Persistent storage	Store accepted telemetry records in PostgreSQL.	Implemented and observed working during development.
Dashboard	Provide a software interface for inspecting monitoring data.	Streamlit dashboard included in the prototype software.
Advanced anomaly detection	Identify abnormal behaviour using a trained machine-learning model.	A planned enhancement; it must not be presented as a fully validated operational feature without further evidence.

1.3 Objectives

The primary objective of Secure-BMS is to develop a working prototype that demonstrates the integration of embedded battery monitoring, local alert-related hardware, secure telemetry processing, persistent data storage, and software-based observation.

The specific objectives are listed below.

1.3.1 Primary Objective

To design and implement a cyber-physical battery monitoring prototype for a physical 18650 lithium-ion cell that combines temperature measurement, embedded processing, secure telemetry, backend validation, persistent storage, and dashboard-based monitoring.

1.3.2 Specific Objectives

1. Develop a temperature-monitoring subsystem.
Integrate an NTC thermistor circuit with an ADS1115 analogue-to-digital converter and ESP32 microcontroller. Convert the acquired analogue measurement into an estimated temperature using the thermistor characteristics used in the prototype.

2. Improve measurement stability through filtering.
Implement median filtering using a group of nine ADC samples, with a 10 ms delay between successive samples, to reduce the influence of isolated fluctuations in the acquired readings.

3. Implement operating-state and sensor-fault logic.
Evaluate the acquired measurements and generate defined telemetry states, including `NORMAL`, `TEMPERATURE_ALERT`, and `SENSOR_FAULT`. Maintain selected monitoring statistics, including peak temperature, minimum and maximum voltage fields where available in the telemetry implementation, alert count and duration, and sensor-fault count. The presence of a telemetry field does not by itself establish that every underlying physical quantity has been independently validated.

4. Integrate local alert-related hardware.
Connect the LM339 comparator signal to ESP32 GPIO26 and the active-low buzzer to GPIO25. Include the IRLZ44N MOSFET gate interface on GPIO27 for the load-control portion of the prototype. Distinguish the existence of these hardware interfaces from proof of autonomous, fail-safe battery protection.

5. Establish wireless telemetry communication.
Use Wi-Fi and MQTT to transfer monitoring data from the ESP32 to the software side of the system. Use the `secure-bms/telemetry` topic for the project's telemetry messages.

6. Apply authenticated encryption to telemetry.
Implement AES-128-GCM protection for telemetry payloads. Include a nonce in the encrypted message envelope and use the MQTT topic as additional authenticated data. Configure the receiver to authenticate and decrypt the payload before validating its fields or storing it.

7. Implement backend validation and replay-state tracking.
Validate decoded telemetry and maintain persistent replay-related session and counter state in PostgreSQL. Reject messages that fail authentication, contain invalid data, or violate the implemented replay rules. Verify these behaviours through isolated tests and identify the live end-to-end replay scenario as a separate testing requirement.

8. Implement persistent telemetry storage.
Use a PostgreSQL database to retain accepted telemetry records and support subsequent access by the software monitoring components.

9. Provide a dashboard for monitoring.
Use Streamlit as the dashboard framework for displaying information obtained from the project's software and data-storage components.

10. Establish a basis for future anomaly analysis.
Organise the monitoring architecture so that stored telemetry can support subsequent analysis. Machine-learning-based anomaly detection, including the proposed use of Isolation Forest and Random Forest methods, is a future or separately validated capability unless its complete execution and results are demonstrated with supporting evidence.

11. Evaluate the prototype through documented tests.
Record the hardware checks, communication behaviour, database operations, and security-

related tests actually performed. Clearly separate component-level tests, isolated logic tests, integration tests, and future validation work.

The relationship between the objectives and their present status is summarised in Table 1.2.

Table 1.2: Project objectives and implementation status

Objective	Evidence available	Status
Temperature sensing	ESP32, ADS1115, and thermistor readings were exercised during development.	Implemented and tested at prototype level
Median filtering	Nine-sample median routine in firmware.	Implemented
State and fault handling	Firmware defines the three reported operating states.	Implemented
Comparator and buzzer interface	LM339 signal and active-low buzzer connected to the specified GPIOs.	Implemented; safety performance not certified
MOSFET load-control interface	IRLZ44N gate connected to GPIO27.	Interface implemented; automated switching not claimed as enabled
Wi-Fi and MQTT telemetry	Firmware publishes telemetry to the configured MQTT topic.	Implemented
AES-128-GCM protection	Encrypted envelope, nonce handling, topic-based authenticated data, and backend decryption.	Implemented
Backend replay-state logic	Isolated tests cover fresh, increasing, duplicate, stale, new-session, and previously seen-session cases.	Implemented; isolated tests passed
Live MQTT replay integration test	No verified end-to-end replay test through the live callback is available.	Pending
PostgreSQL telemetry persistence	Accepted telemetry records were observed being saved during development.	Implemented
Streamlit dashboard	Dashboard component is part of the software prototype.	Implemented
Validated ML anomaly detection	No complete, verified model-evaluation results are established here.	Planned or requiring further validation
Production-grade battery safety	No certification or comprehensive safety validation has been established.	Outside current demonstrated scope

1.4 Scope and Limitations

This section defines the functions included in the Secure-BMS prototype and identifies the boundaries within which its implementation and results should be interpreted. The distinction is important because a functional educational prototype does not automatically provide the same assurance as a commercial BMS that has undergone extensive electrical, environmental, reliability, and safety testing.

Scope

The scope of Secure-BMS includes the development and integration of the following components.

1. Physical battery and temperature sensing

The prototype uses a physical 18650 lithium-ion cell as the battery under study. Temperature-related measurements are obtained using an approximately 100 kΩ B3950 NTC thermistor circuit, a 10 kΩ divider resistor, an ADS1115 analogue-to-digital converter, and an ESP32 microcontroller. The implementation focuses on temperature measurement and its use in the monitoring logic.

2. Embedded firmware and measurement processing

The ESP32 firmware performs sensor acquisition, median filtering, temperature estimation, fault checks, operating-state evaluation, and alert-related output handling. It also prepares telemetry and tracks selected statistics. The firmware includes Wi-Fi reconnection handling and uses the configured GPIO interfaces for the comparator signal, buzzer, and MOSFET gate.

3. Local alert-related hardware

The system incorporates an LM339 comparator-based signal and an active-low buzzer interface. These components provide local alert-related behaviour in addition to software monitoring. A MOSFET-based load-control interface is also included. The scope is limited to the implemented prototype arrangement; the report does not claim that it provides certified overtemperature protection, guaranteed disconnection under every fault, or an independently validated emergency shutdown mechanism.

4. Wireless telemetry

The ESP32 communicates monitoring information through Wi-Fi and MQTT. The MQTT telemetry topic used by the project is `secure-bms/telemetry`. The receiving software processes incoming messages and handles the encrypted telemetry envelope.

5. Secure message processing

The communication-security implementation uses AES-128-GCM authenticated encryption. The encrypted envelope carries an encryption identifier, a Base64-encoded nonce, and Base64-encoded ciphertext with the authentication tag. The topic is used as additional authenticated data. The backend authenticates and decrypts incoming messages before validating the decoded telemetry.

The firmware's nonce design combines an eight-byte session prefix generated for a boot session with a four-byte big-endian counter. The backend stores replay-related session and counter information in PostgreSQL and checks incoming messages against that state. Isolated tests of the replay-state logic have passed. Testing the complete behaviour through the live MQTT callback remains an outstanding validation task.

6. Backend and database

The software backend uses Python, FastAPI, MQTT message handling, and PostgreSQL. Incoming telemetry is validated and accepted records are stored in the database. Replay-related state is maintained separately from the primary telemetry records. This enables the backend to evaluate message freshness as well as process monitoring information.

7. Dashboard and monitoring interface

The project includes a Streamlit dashboard for viewing monitoring information made available by the software system. The scope is software-based observation of the prototype; the dashboard should not be treated as a substitute for physical instrumentation or as a certified safety-control interface.

8. Testing and evaluation

The scope includes documenting the tests performed on the sensing hardware, comparator and buzzer interfaces, telemetry processing, database storage, and replay-state logic. The report distinguishes directly observed hardware behaviour from isolated software tests and from integration tests that remain pending.

9. Future analytical extensions

The stored telemetry provides a basis for future data analysis and anomaly-detection experiments. Isolation Forest and Random Forest have been considered in the proposed project architecture, but their presence in the planned technology stack does not establish that a trained model is operational, validated, or producing reliable predictions. Such capabilities are included as future work unless supported by implementation and evaluation evidence.

The main scope boundaries are summarised in Table 1.3.

Table 1.3: Scope of Secure-BMS

Area	Included in the current prototype	Boundary
Battery	Physical 18650 lithium-ion cell	No claim of compatibility with all cell types or battery packs
Temperature sensing	NTC thermistor, ADS1115, ESP32	Accuracy and calibration across the full operating range require further characterisation
Embedded processing	Filtering, temperature estimation, state and fault logic	Does not constitute a complete commercial BMS
Local alert hardware	Comparator signal and buzzer interface	No certified safety or fail-safe guarantee
Load interface	ESP32-controlled MOSFET gate interface	Automated load switching is not claimed as active while `LOAD_ENABLE = false`
Communication	Wi-Fi and MQTT telemetry	Availability depends on the configured network and broker
Message security	AES-128-GCM authenticated encryption and backend verification	Overall security depends on key management, deployment configuration, and further security testing

Replay protection	Persistent session and counter checks	Isolated tests passed; live end-to-end replay integration test remains pending
Storage	PostgreSQL telemetry and replay-state tables	Retention, backup, and production-scale performance are not established
Dashboard	Streamlit monitoring interface	Not a replacement for local protection or independent measurement equipment
Machine learning	Future analysis of telemetry	Operational model performance is not established without validated experiments
Deployment	Educational prototype	No commercial certification or production deployment claim

Limitations

The current implementation has several limitations that must be considered when interpreting its results.

1. Limited measurement coverage

The demonstrated sensing subsystem focuses primarily on temperature. Although the firmware includes selected voltage-related telemetry fields and statistics, the availability of those fields should not be interpreted as proof of calibrated voltage measurement across the operating range. Current measurement through the INA219 was not fully validated and was intentionally set aside during the development process. Consequently, comprehensive current monitoring and power measurement are not established capabilities of the current prototype.

2. Sensor accuracy and calibration

The temperature estimate depends on the thermistor's characteristics, the voltage-divider arrangement, ADC behaviour, component tolerances, and the assumptions used by the firmware. Initial readings and temperature changes were exercised during development, but this does not establish traceable calibration or a quantified accuracy specification across the full operating range. A future validation phase should compare the readings with a suitable reference thermometer under controlled conditions.

3. Limited battery-management functions

The prototype does not demonstrate a complete commercial battery-management feature set. Comprehensive state-of-charge and state-of-health estimation, cell balancing, pack-level monitoring, and validated current-limit enforcement are not established by the present implementation. The project should therefore be described as a battery monitoring and protection-oriented testbed rather than a fully featured BMS.

4. Load-control limitations

The MOSFET gate interface is connected to the ESP32, but the firmware configuration has `LOAD_ENABLE = false`. Accordingly, the report must not claim that autonomous load switching or automated battery disconnection has been demonstrated unless subsequent implementation and test evidence establishes it. A physical load-control interface alone does not prove that every fault condition results in a safe disconnection.

5. Network dependency

Telemetry delivery and remote viewing depend on the ESP32's Wi-Fi connection, MQTT broker availability, and operation of the receiving software. The firmware includes Wi-Fi reconnection handling, but this does not establish uninterrupted communication or guaranteed delivery during every network outage. Local embedded behaviour and remote monitoring should be evaluated as separate functions.

6. Security validation limits

AES-128-GCM authentication and decryption are implemented in the message-processing path. Replay-state logic has been tested using isolated unit and database-level scenarios, including acceptance of a new session and rejection of duplicate, stale, and previously seen session data. These tests support the correctness of the isolated replay checks but do not establish that the complete live MQTT path has passed an end-to-end replay test.

Further testing is needed to evaluate malformed envelopes, authentication failures, counter boundaries, device restarts, concurrent message processing, key protection, and recovery from database or network failures. The use of authenticated encryption also does not, by itself, establish secure key provisioning, secure firmware updates, or complete system-wide cybersecurity.

7. Machine-learning validation

Machine-learning-based anomaly detection is part of the proposed extension of the architecture. However, without verified training data, a completed model pipeline, and documented evaluation results, the report cannot claim a particular detection accuracy or demonstrate that the system reliably predicts battery faults. Model development and evaluation should be treated as future work until supported by reproducible evidence.

8. Limited test duration and environmental coverage

Prototype testing has established that selected components and software paths operate during development. It has not established reliability over long-duration operation, a wide range of ambient temperatures, repeated charge-discharge cycles, component ageing, or all possible fault conditions. These limitations restrict the conclusions that can be drawn from the current results.

9. No formal safety certification

Secure-BMS is an academic prototype. It has not been established as compliant with a battery safety certification regime, nor has it undergone a complete independent safety assessment. Testing the prototype must therefore be conducted with appropriate precautions and should not be interpreted as evidence that the system is suitable for unattended or safety-critical battery operation.

10. Deployment and scalability

The current system is developed around a specific prototype configuration and software environment. Performance under large numbers of devices, high telemetry rates, prolonged

database operation, or deployment across multiple sites has not been established. Production deployment would require additional work on configuration management, monitoring, backup and recovery, access control, key management, and operational maintenance.

In summary, Secure-BMS demonstrates an integrated prototype for temperature-oriented battery monitoring, embedded state evaluation, alert-related hardware, authenticated MQTT telemetry, backend validation, replay-state tracking, persistent storage, and dashboard-based observation. Its contribution lies in combining these functions in an experimentally accessible cyber-physical system. The conclusions of the report must remain limited to functions that have been implemented and supported by actual test evidence, while unverified integration behaviour and proposed analytical enhancements are explicitly identified as future work.

2. Literature Survey / Existing System

Battery Management Systems (BMS) are important components of battery-powered applications because they monitor battery operating conditions and support the protection, performance, and reliability of rechargeable cells and battery packs. Traditional BMS architectures primarily focus on electrical measurement, temperature monitoring, threshold-based fault detection, and protective control. More recent approaches extend these functions using Internet of Things (IoT) communication, cloud-based data storage, remote dashboards, and data-driven analytical methods.

The development of Secure-BMS draws upon these established areas. The project combines embedded temperature monitoring and local alert-related hardware with wireless telemetry, authenticated message processing, persistent storage, and a dashboard. The literature review therefore considers four areas: conventional battery management systems, IoT-based battery monitoring, cloud-connected battery monitoring, and machine-learning-based anomaly detection.

The purpose of this review is to understand the capabilities of existing approaches, identify engineering considerations relevant to the proposed prototype, and establish the rationale for integrating these functions into a single system. It does not assume that every existing BMS lacks security, remote monitoring, or advanced analytics. Instead, it identifies the particular integration and validation requirements relevant to the scope of Secure-BMS.

2.1 Review of Existing Systems

Existing battery monitoring and management solutions can be examined according to their sensing capabilities, processing architecture, protection mechanisms, communication methods, and analytical functions. Some systems are designed primarily for local electrical protection, whereas others provide remote observation, long-term data collection, or advanced state estimation.

Gabbar, Othman, and Abdussami [1] review BMS development and industrial standards, discussing monitoring, protection, battery-state estimation, and system architecture. Their review establishes that battery management involves more than displaying sensor measurements: a BMS may also need to estimate battery condition and initiate appropriate protective actions.

Similarly, the review by Brandl et al. and related battery-system safety literature emphasise the importance of measurement reliability and the consequences of inadequate monitoring. Accurate acquisition of battery voltage, current, and temperature is fundamental to subsequent state estimation and fault detection [2].

These studies provide the foundation for comparing existing approaches with Secure-BMS. The following subsections examine the principal categories relevant to the project.

2.1.1 Conventional Battery Management Systems

A conventional Battery Management System monitors relevant electrical and thermal parameters and uses them to determine whether the battery is operating within defined limits. Depending on the application, its functions may include cell-voltage monitoring, current measurement, temperature measurement, state-of-charge estimation, state-of-health estimation, cell balancing, and protective disconnection.

A typical conventional architecture contains sensors or analogue measurement circuits, a processing unit, decision-making logic, and output devices. Measurements are acquired from the battery, processed by the controller, and compared against predefined limits or model-based criteria. If an abnormal condition is detected, the BMS may generate an alert, restrict operation, or activate a protective mechanism.

The principal characteristics of a conventional BMS are summarised below.

1. Parameter measurement

Voltage, current, and temperature are commonly monitored because they provide essential information about battery operating conditions. Measurement circuits must be selected according to the battery chemistry, operating range, required precision, and intended safety function.

2. Threshold-based monitoring

A threshold-based system compares a measured parameter with a predefined limit. For example, a temperature measurement exceeding a configured threshold may cause the system to enter an alert state. This approach is comparatively straightforward to implement and explain, making it useful in embedded applications.

However, threshold-based logic depends on appropriate threshold selection, reliable measurements, and correct implementation. A fixed threshold does not independently predict degradation or identify every abnormal behaviour that may occur within the nominal operating range.

3. Local protection

Conventional BMS designs may use relays, contactors, MOSFETs, or dedicated protection circuits to control the battery's connection to a load or charger. Such mechanisms must be designed and validated for the relevant voltage, current, thermal, and fault conditions.

4. Battery-state estimation

More advanced systems estimate quantities such as state of charge (SoC) and state of health (SoH). These estimates require suitable models, measurement inputs, and validation. They cannot be inferred reliably from a temperature reading alone.

5. Embedded processing

A local controller can evaluate measurements and execute protective logic without requiring a cloud service for every decision. This is an important architectural consideration for systems in which network availability cannot be guaranteed.

Conventional BMS designs provide a foundation for dependable local monitoring. Nevertheless, a basic threshold-oriented implementation may not include long-term remote data access, authenticated telemetry, persistent replay tracking, or machine-learning-based analysis in the same system. These capabilities depend on the specific design and are not universally absent from conventional commercial products.

For Secure-BMS, the conventional approach provides the basis for local temperature measurement, state evaluation, and alert-related hardware. The prototype uses an ESP32, an ADS1115 analogue-to-digital converter, an NTC thermistor circuit, an LM339 comparator signal, and an active-low buzzer interface. Its firmware includes median filtering and checks for defined sensor-fault and temperature-alert conditions.

The current implementation is narrower than a complete commercial BMS. In particular, comprehensive cell balancing, validated SoC and SoH estimation, and fully characterised electrical protection are not established features of the prototype. Its contribution is to demonstrate the integration of selected embedded monitoring functions with secure networked telemetry and software-based observation.

2.1.2 IoT-Based Battery Monitoring Systems

The Internet of Things (IoT) enables physical devices to transmit sensor measurements to other devices, local servers, or remote software platforms. In battery monitoring, IoT connectivity can make measurements available beyond the physical battery enclosure and allow operators to inspect system behaviour through a computer or mobile interface.

IoT-based battery monitoring generally involves the following stages:

1. **Data acquisition:** sensors measure selected battery parameters.
2. **Embedded processing:** a microcontroller reads the measurements and may perform filtering or threshold checks.
3. **Communication:** the device transmits data using a network protocol.
4. **Data reception:** a broker or receiving application accepts the messages.
5. **Presentation or storage:** a dashboard displays the measurements, or a database retains them for later analysis.

An example is the work titled *Development of Battery Monitoring System in Smart Microgrid Based on Internet of Things (IoT)* [3]. The study describes a monitoring architecture that combines battery-system data acquisition, communication with a cloud system, and a human-machine interface. It demonstrates how IoT technologies can extend battery monitoring beyond a local measurement unit.

Another example is the 2026 conference publication *Battery Management System with IoT Based Monitoring and Data Logging* [4]. Its described architecture uses an ESP32-based monitoring arrangement, records measurements through cloud-connected services, and includes local display and fault-related control features. This illustrates that IoT battery monitoring can combine local sensing with remote data access.

Message Queuing Telemetry Transport (MQTT) is particularly relevant to systems that exchange relatively small telemetry messages. MQTT uses a publish-subscribe communication model in which a publisher sends a message to a topic and a broker distributes that message to subscribed clients. This separates the embedded device from the receiving application's implementation and allows multiple software components to consume the same telemetry stream when configured appropriately.

For a battery monitoring system, MQTT can be used to transmit information such as temperature estimates, operating states, and selected diagnostic statistics. The publisher does not need to know the internal structure of the dashboard or database, provided that the receiver follows the agreed message format and topic conventions.

IoT integration also introduces additional engineering requirements. A monitoring device must handle network disconnections, malformed messages, communication failures, and the possibility that data may be delayed or repeated. Remote availability is not equivalent to guaranteed local protection. For this reason, the design should distinguish embedded decision-making from the transmission and presentation of telemetry.

In Secure-BMS, Wi-Fi and MQTT are used to transmit monitoring information from the ESP32 to the software backend. The topic `secure-bms/telemetry` is used for telemetry messages. The firmware includes Wi-Fi reconnection handling, while the backend processes the incoming message before the accepted data is saved to PostgreSQL.

The Secure-BMS implementation also differs from a simple arrangement in which sensor values are sent as unauthenticated JSON. It incorporates AES-128-GCM authenticated encryption, backend decryption, and payload validation. Persistent replay-state tracking has been implemented separately. These features address message authenticity and freshness concerns that become relevant when monitoring data travels through a network.

The principal benefits and considerations of IoT-based battery monitoring are presented in Table 2.1.

Table 2.1: Characteristics of IoT-based battery monitoring

Characteristic	Benefit	Engineering consideration
Wireless telemetry	Allows measurements to be accessed beyond the physical device.	Depends on network connectivity and correct communication configuration.
Publish-subscribe messaging	Separates the publisher from receiving applications.	Topics, payload schemas, and subscriber behaviour must be managed consistently.
Remote alerts	Can make abnormal conditions visible to an operator remotely.	Remote notification must not replace required local protective behaviour.

Multiple software consumers	Enables monitoring data to be used by storage, dashboards, or analytics.	Data validation and consistent interpretation are required across components.
Network reconnection	Supports recovery after some connection interruptions.	Reconnection does not guarantee delivery of every message.
Security controls	Can protect telemetry from unauthorised modification or acceptance.	Authentication, key management, access control, and replay handling require explicit design.

IoT-based monitoring therefore provides the communication foundation for Secure-BMS. The project extends this foundation with authenticated telemetry and persistent backend processing while retaining embedded monitoring and local alert-related hardware.

2.1.3 Cloud-Connected Battery Monitoring

Cloud-connected battery monitoring extends an IoT architecture by storing measurements or processing results in a database or remotely accessible computing environment. This allows an operator to inspect recent measurements, review historical trends, and potentially combine information from multiple devices.

A cloud-connected monitoring architecture commonly includes a device layer, a communication layer, a processing or service layer, a storage layer, and a user-interface layer. These components may run on remote cloud infrastructure or on a local computer or server. The term *cloud-connected* describes the architectural separation between the monitoring device and the data-processing services; it does not necessarily mean that every component must be hosted by a commercial cloud provider.

Chen et al. [5] describe an online battery monitoring and management system based on IoT. Their study illustrates the use of connected monitoring and fault-diagnosis concepts in battery management. Such architectures support the broader transition from isolated measurements to connected systems in which data can be retained and processed beyond the embedded controller.

Persistent storage is an important component of this architecture. A database can retain accepted measurements together with associated fields and timestamps, allowing the system to retrieve historical information instead of relying only on the latest reading displayed by the device. Stored data can also support later analysis, although the usefulness of that analysis depends on measurement quality, data completeness, and appropriate processing.

A dashboard provides a user-facing layer for accessing this information. Depending on its design, it may display current operating states, measurement trends, fault indicators, or historical records. The dashboard is primarily a presentation and interaction component; it should not be assumed to perform the embedded device's protective functions unless a separately designed and validated control path exists.

Cloud-connected systems also introduce several considerations:

- **Availability:** the receiving service and database must be accessible for remote monitoring to work.

- **Data integrity:** received messages should be validated before being stored or displayed.
- **Security:** communication and stored information may require authentication, access control, and encryption.
- **Consistency:** the message schema must remain consistent across firmware, backend, and dashboard components.
- **Historical data management:** retention, indexing, backup, and recovery become important as the volume of stored measurements increases.
- **Operational independence:** local monitoring and protection should be considered separately from remote dashboard availability.

Secure-BMS adopts a layered software arrangement. The ESP32 publishes telemetry over Wi-Fi and MQTT. A Python backend processes incoming messages, uses AES-GCM authentication and decryption, validates the decoded data, and stores accepted telemetry in PostgreSQL. FastAPI provides the backend API framework, and Streamlit is used for the dashboard interface.

The project uses a PostgreSQL database named `secure_bms`. In addition to the telemetry data, separate tables are used to maintain replay-related session and counter state. This separates the monitoring records from the information required to determine whether a received encrypted message is new or has already been observed.

The project should be described accurately as a prototype with a connected software monitoring architecture. A working local MQTT broker and local database do not, by themselves, demonstrate deployment on a public cloud platform. Similarly, a dashboard that displays monitoring information does not establish that all data is available continuously or that the system has been tested under large-scale deployment conditions.

The comparison between a conventional local monitoring arrangement and a connected monitoring arrangement is shown in Table 2.2.

Table 2.2: Local and connected battery monitoring architectures

Aspect	Local monitoring architecture	Cloud-connected or server-connected architecture
Primary processing	Performed mainly on the embedded controller.	Shared between the embedded controller and receiving software services.
Data visibility	Usually available through local indicators or interfaces.	Can be presented through a separate dashboard.
Historical records	Depend on local storage or external logging.	Can be maintained in a database for later retrieval.
Network dependency	Local functions may operate without a network.	Remote viewing and data reception depend on communication and service availability.
Security considerations	Includes physical access and firmware-related concerns.	Also includes message authentication, network access, service security, and data validation.
Analytical extensions	Limited by local processing and available storage.	Stored data can support additional processing and analysis.

Relevance to Secure-BMS	Provides the basis for embedded sensing and alert logic.	Provides the MQTT, backend, PostgreSQL, and Streamlit monitoring layers.

The principal contribution of this architecture to Secure-BMS is the ability to connect physical measurements with a software-based data-processing and storage workflow. Its security and reliability must nevertheless be evaluated through testing of the complete communication path rather than inferred from the presence of individual components.

2.1.4 Machine-Learning-Based Anomaly Detection

Rule-based monitoring evaluates measurements using predefined thresholds, logical conditions, or explicit fault criteria. Such methods are useful when the expected operating limits are known and the system must respond to a clearly defined condition. However, unusual behaviour does not always correspond to a single parameter crossing a fixed threshold.

Machine-learning-based anomaly detection offers a complementary approach. An anomaly-detection model examines measured features or learned patterns and identifies observations that differ from the behaviour represented in its training data or reference distribution. Depending on the method, the model may operate without requiring a complete set of labelled fault examples.

In battery applications, potential inputs for analytical models include temperature, voltage, current, rates of change, and other features derived from sensor measurements. The suitability of any input depends on whether it is reliably measured, available at the required frequency, and relevant to the anomaly being investigated. A model cannot reliably compensate for missing or unvalidated measurements simply because a machine-learning algorithm is used.

The broader BMS literature reviews data-driven estimation and modelling approaches alongside conventional measurement and protection methods [6]. Such research provides motivation for exploring analytics as an extension of monitoring systems. However, the usefulness of a model depends on the data, the selected features, the model configuration, and the evaluation procedure.

Two algorithms considered for the proposed Secure-BMS architecture are Isolation Forest and Random Forest.

Isolation Forest

Isolation Forest is an unsupervised anomaly-detection method that identifies observations that can be isolated relatively quickly through random partitioning of the feature space. It can be used to assign anomaly-related scores to observations without requiring every training example to be labelled as normal or faulty.

For battery telemetry, an Isolation Forest model could be explored using validated temperature and other available measurements. The resulting scores would require evaluation against representative operating data and known test scenarios before they could be used as meaningful indicators of battery anomalies.

Random Forest

Random Forest is an ensemble-learning method that combines predictions from multiple decision trees. It can be used for classification or regression when a suitable dataset and target variable are available. For supervised fault classification, for example, the training dataset would require meaningful labels for the fault categories being predicted.

A Random Forest classifier could be explored to classify defined operating conditions if sufficient representative, labelled data were collected. Its performance would need to be measured using suitable evaluation methods and a test set that is separate from the training data.

The two algorithms are not interchangeable. Isolation Forest is commonly used for unsupervised anomaly detection, whereas Random Forest is typically used for supervised classification or regression, although their suitability depends on the exact task. Neither algorithm automatically provides a safety guarantee, and a model's output should not be treated as a substitute for validated protective logic.

Machine-learning-based monitoring also introduces practical challenges:

1. **Data quality:** sensor faults and measurement noise can be misinterpreted as unusual physical behaviour.
2. **Representative training data:** normal operating conditions and relevant fault scenarios must be represented appropriately.
3. **Feature selection:** the model must use measurements that are meaningful and sufficiently reliable.
4. **False positives:** normal variation may be incorrectly identified as anomalous.
5. **False negatives:** dangerous or unusual conditions may not be identified by the model.
6. **Evaluation:** performance must be measured on data that was not used to train or tune the model.
7. **Interpretability:** the model output must be translated into understandable information for operators.
8. **Safety integration:** any action based on an analytical result must be separately designed and validated.

For Secure-BMS, machine-learning-based anomaly detection is an intended extension of the monitoring and storage architecture. The project has a telemetry-processing path and PostgreSQL storage that can provide a foundation for later analysis. However, a fully operational, trained, and validated Isolation Forest or Random Forest model has not been established by the implementation evidence considered in this report. Therefore, no detection accuracy, false-positive rate, or predictive performance is claimed in this chapter.

Table 2.3 compares threshold-based monitoring with machine-learning-based anomaly detection.

Table 2.3: Comparison of threshold-based and machine-learning-based monitoring

Aspect	Threshold-based monitoring	Machine-learning-based anomaly detection
Decision	Compares measurements with	Uses a learned model or statistical

mechanism	predefined conditions.	pattern to identify unusual observations.
Data requirements	Requires appropriate limits and reliable measurements.	Requires suitable data for training, configuration, and evaluation.
Interpretability	Often straightforward because the triggering condition is explicit.	Depends on the model and the way its output is explained.
Adaptability	Changes require threshold or rule adjustment.	May adapt to represented patterns through retraining or model updates.
Failure modes	Can miss behaviour that does not cross a configured threshold.	Can miss anomalies not represented or detectable in the model's feature space.
False alarms	Depend on threshold choice, noise, and measurement quality.	Depend on model configuration, data distribution, and decision criteria.
Role in Secure-BMS	Implemented for defined states and fault checks.	Proposed for future analytical extensions; performance requires validation.

A practical design can use both approaches for different purposes. Explicit threshold and fault logic can handle defined operating conditions, while a separately validated analytical model may help identify unusual patterns in historical or incoming telemetry. In Secure-BMS, this combined approach remains a development direction rather than a claim that machine-learning-based battery fault prediction has already been demonstrated.

2.2 Gaps Identified and Proposed Solution

The literature reviewed in Sections 2.1.1–2.1.4 demonstrates that battery monitoring can involve local measurement and protection, IoT communication, remote data storage, and advanced analytical techniques. These capabilities can be implemented independently or integrated into a larger system. The central design challenge for Secure-BMS is to integrate selected capabilities while maintaining clear boundaries between sensing, local state evaluation, communication security, persistent storage, and software presentation.

The following gaps are defined in relation to the scope and objectives of the proposed prototype. They should be interpreted as requirements for this project rather than universal shortcomings of existing commercial or research systems.

2.2.1 Gaps Identified

1. Integration of physical sensing and software monitoring

A standalone sensor circuit can provide measurements, while a separate software application can store and display data. However, an integrated project must establish a consistent data path between the physical device and the software components.

Secure-BMS addresses this integration requirement by connecting the thermistor and ADC measurement path to the ESP32 firmware, transmitting telemetry through MQTT, processing incoming messages in the backend, storing accepted records in PostgreSQL, and making monitoring information available through a Streamlit dashboard.

2. Reliable processing of temperature measurements

Temperature monitoring depends on the quality of the acquired signal and the logic used to interpret it. Isolated fluctuations may produce unstable estimates if raw readings are processed without filtering. Sensor faults must also be distinguished from valid measurements.

The prototype incorporates median filtering using nine ADC samples, with 10 ms delays between samples, together with defined sensor-fault checks. Further calibration and controlled testing are still required to quantify measurement accuracy across the intended operating range.

3. Separation of local alert handling from remote monitoring

A connected dashboard is useful for observation, but the availability of a remote service depends on communication and software components that may fail independently of the battery hardware.

Secure-BMS includes embedded state evaluation and alert-related hardware in addition to its remote monitoring path. This addresses the architectural requirement to keep local monitoring behaviour distinct from remote presentation. It does not establish certified or fail-safe battery protection, and the load-control interface requires further validation before autonomous disconnection can be claimed.

4. Authentication of telemetry messages

A receiving application should not assume that every message arriving through a communication broker is trustworthy. Message contents may be malformed or modified, and the receiver must be able to determine whether an encrypted message has passed authentication.

Secure-BMS uses AES-128-GCM authenticated encryption. The backend decrypts and authenticates the encrypted envelope before validating the decoded telemetry. The MQTT topic is used as additional authenticated data, binding the authentication operation to the expected topic.

This design addresses message authenticity and integrity at the payload-processing level. It does not, by itself, establish comprehensive security across device provisioning, credential storage, network access, firmware updates, or deployment configuration.

5. Detection of repeated or stale telemetry

A message may be authentic but still be old or repeated. Consequently, message authentication and replay protection address different concerns. A complete receiver needs a defined way to identify whether an incoming message represents a new telemetry event or repeats a previously accepted sequence.

The Secure-BMS backend maintains persistent replay-related state using an eight-byte session identifier and a four-byte counter extracted from the twelve-byte AES-GCM nonce.

The implemented logic checks counter progression within a session and rejects previously observed sessions according to the project's replay rules.

Isolated unit and database-level tests have passed for fresh messages, increasing counters, duplicate counters, stale counters, new sessions, and previously seen sessions. However, an end-to-end replay attack through the live MQTT callback has not yet been verified. This remains a specific integration-testing requirement.

6. Persistent and structured telemetry records

Remote monitoring becomes more useful when accepted measurements can be retrieved after the original message has been processed. Storing data in a structured database also provides a foundation for future analysis and review.

Secure-BMS uses PostgreSQL for telemetry persistence and maintains replay state in separate tables. The backend saves accepted telemetry records only after the message-processing and validation steps have been completed. Long-term retention policies, backup and recovery, and production-scale performance require further evaluation.

7. Separation of backend processing and dashboard presentation

Combining message reception, data validation, storage, and presentation into a single tightly coupled component can make the system harder to maintain and test. A layered design provides clearer responsibilities and allows each part to be examined independently.

Secure-BMS uses a Python backend with MQTT message processing and FastAPI, PostgreSQL for storage, and Streamlit for the dashboard. This separation creates identifiable interfaces between the embedded device, communication layer, backend, database, and user interface.

8. A foundation for future data-driven analysis

Threshold-based rules are suitable for explicit conditions, but historical telemetry may also contain patterns worth investigating through statistical or machine-learning methods. Such analysis requires reliable measurements and a suitable evaluation process.

Secure-BMS provides a data-storage foundation for future analytical work. Isolation Forest and Random Forest are proposed as possible methods, but a completed and validated anomaly-detection pipeline has not been established here. Their integration, dataset preparation, and performance evaluation remain future work.

The identified gaps and the corresponding project responses are summarised in Table 2.4.

Table 2.4: Identified gaps and Secure-BMS responses

Identified gap or requirement	Proposed system response	Current status
Integration between physical sensing and software monitoring	Connect ESP32 sensing and telemetry to MQTT, backend processing, PostgreSQL, and	Implemented at prototype level

	Streamlit.	
Fluctuating analogue readings	Apply nine-sample median filtering.	Implemented
Defined temperature-alert and sensor-fault handling	Evaluate readings and produce the configured operating states.	Implemented
Need for local alert-related hardware	Include comparator signal and active-low buzzer interface.	Implemented and exercised at prototype level
Need for a load-control interface	Connect an IRLZ44N MOSFET gate to the ESP32.	Interface implemented; automated load operation not claimed while disabled
Risk of accepting unauthenticated telemetry	Use AES-128-GCM and validate decrypted payloads.	Implemented
Risk of repeated or stale messages	Track session identifiers and counters in PostgreSQL.	Logic implemented; isolated tests passed
Need to retain accepted telemetry	Store validated telemetry in PostgreSQL.	Implemented
Need for accessible monitoring information	Provide a Streamlit dashboard.	Implemented
Need for broader anomaly analysis	Explore Isolation Forest and Random Forest using validated telemetry.	Planned or requiring further validation
Need for full integration evidence	Test the entire telemetry path, including live replay attempts and recovery cases.	Additional tests pending

The table demonstrates that the project has addressed several integration requirements in its current prototype. It also identifies the distinction between implemented functions and those requiring further validation. This distinction is essential for interpreting the results in later chapters.

2.2.2 Proposed Solution: Secure-BMS

Secure-BMS proposes a layered cyber-physical architecture that connects a physical lithium-ion cell and its sensing hardware to embedded processing, secure wireless telemetry, backend validation, persistent storage, and a dashboard. Its purpose is to demonstrate the integration of these components in a manageable prototype rather than to reproduce the full functionality of an industrial BMS.

The proposed solution is organised into the following layers.

1. Physical sensing layer

The physical layer contains the 18650 lithium-ion cell and the temperature-sensing arrangement. An NTC thermistor with approximately 100 kΩ nominal resistance and B3950 characteristics is used in a voltage-divider circuit with a 10 kΩ resistor. The ADS1115 ADC acquires the analogue signal for processing by the ESP32.

This layer provides the physical measurements used by the embedded monitoring logic. The current implementation focuses on temperature-oriented monitoring; comprehensive validated voltage and current measurement is outside the established scope.

2. Embedded processing and local alert layer

The ESP32 executes the firmware responsible for acquiring and processing measurements. The firmware applies median filtering, estimates temperature, checks for sensor-related faults, evaluates the defined operating states, and prepares telemetry.

The local hardware interfaces include an LM339 comparator signal on GPIO26, an active-low buzzer on GPIO25, and an IRLZ44N MOSFET gate on GPIO27. The design combines software state evaluation with separate alert-related hardware. However, the prototype's load-control configuration must be described accurately: with `LOAD_ENABLE = false`, autonomous load switching is not claimed as an active demonstrated function.

3. Wireless communication layer

The ESP32 uses Wi-Fi to communicate with the MQTT broker. Monitoring information is published using the `secure-bms/telemetry` topic. MQTT separates the device's publishing function from the receiving software components.

The communication layer provides the transport mechanism for telemetry. It is not assumed to guarantee uninterrupted connectivity, and network availability is not treated as a prerequisite for every local embedded operation.

4. Security and backend processing layer

The embedded system encrypts telemetry using AES-128-GCM. The message envelope includes the encryption identifier, nonce, and encrypted payload with its authentication tag. The MQTT topic is used as additional authenticated data.

The backend authenticates and decrypts the incoming payload, validates the decoded fields, and passes accepted data to the database layer. It also implements replay-state tracking using a persistent session and counter record. This design distinguishes the authenticity of a message from its freshness.

The replay-state logic has passed isolated unit and database-level tests. Full end-to-end testing of replay rejection through the live MQTT message callback remains pending and must be reported as such.

5. Persistent storage layer

PostgreSQL stores accepted telemetry and the separate replay-state information used by the backend. Persistent storage makes it possible to retain records beyond the processing of an individual MQTT message.

The database layer provides a foundation for retrieving monitoring information and supporting later analysis. It does not automatically establish long-term data completeness, production reliability, or backup and recovery performance.

6. Dashboard and monitoring layer

The Streamlit dashboard provides a software interface for inspecting the project's monitoring information. FastAPI is included in the backend software architecture, while Python-based processing handles MQTT messages and database operations.

The dashboard is intended for observation and analysis. It should not be interpreted as a substitute for physical test instruments or a validated local protection mechanism.

7. Future analytical layer

The stored telemetry can support future experiments involving anomaly detection. Isolation Forest may be explored for unsupervised identification of unusual observations, while Random Forest may be considered for classification or regression tasks when suitable data and target labels are available.

This layer remains a planned analytical extension unless a complete model, dataset, and evaluation results are produced. No predictive accuracy or fault-detection performance is claimed in the present chapter.

Table 2.5 provides an overview of the proposed solution and its implementation status.

Table 2.5: Secure-BMS architecture and implementation status

Layer	Main components or technologies	Responsibility	Status
Physical sensing	18650 cell, NTC thermistor, voltage-divider resistor, ADS1115	Obtain temperature-related measurements.	Implemented and exercised
Embedded processing	ESP32 firmware, median filtering, state and fault logic	Process measurements and prepare telemetry.	Implemented
Local alert-related hardware	LM339 comparator, active-low buzzer	Provide comparator and audible-alert interfaces.	Implemented at prototype level
Load-control interface	IRLZ44N MOSFET, ESP32 GPIO27	Provide the hardware interface for load control.	Interface implemented; automated operation not claimed while disabled
Wireless communication	Wi-Fi, MQTT, `secure-bms/telemetry`	Transmit monitoring messages.	Implemented
Message security	AES-128-GCM, nonce and topic-based authenticated data	Authenticate and decrypt telemetry.	Implemented
Backend processing	Python, MQTT message handling, FastAPI	Validate messages and coordinate processing.	Implemented

Replay protection	PostgreSQL session and counter state	Reject repeated or stale telemetry under the implemented rules.	Implemented; isolated tests passed; live integration test pending
Persistent storage	PostgreSQL	Retain accepted telemetry and replay state.	Implemented
Dashboard	Streamlit	Present monitoring information.	Implemented
Advanced analytics	Isolation Forest, Random Forest	Explore anomaly detection and data-driven classification.	Planned or requiring further validation

The proposed solution is therefore a connected battery-monitoring prototype that combines selected local hardware functions with authenticated telemetry and software-based data management. Its main architectural objective is to establish a traceable path from physical measurement to validated, stored, and viewable information.

The approach also provides a basis for systematic testing. The embedded measurement logic, communication security, database persistence, and dashboard can be evaluated as individual components, while integration testing can establish whether the complete system behaves correctly when messages are delayed, malformed, repeated, or rejected. Additional testing is required before the prototype can be considered suitable for broader battery-safety applications.

In conclusion, the literature review supports the use of a layered architecture that combines embedded monitoring, connected data acquisition, structured storage, and software observation. Secure-BMS implements these selected capabilities while retaining explicit boundaries around measurement validation, autonomous load control, end-to-end replay testing, and machine-learning performance. These boundaries define the scope of the implementation and provide a clear basis for the requirements analysis and system design presented in the subsequent chapters.

3. Software Process Model and Project Planning

Software process planning is an important part of developing a cyber-physical system because the project involves multiple interacting components, including physical hardware, embedded firmware, communication protocols, backend services, database operations, and a dashboard. Changes to one component can affect other components. For example, a change in the firmware telemetry format may require corresponding changes in backend validation and database processing.

Secure-BMS combines hardware experimentation with software development. Its development therefore requires a process that supports incremental implementation, testing, defect correction, and integration across the hardware and software layers. A sequential approach in which every requirement is finalised before implementation may be difficult to

apply when hardware behaviour, sensor readings, and communication issues must be investigated experimentally.

This chapter describes the software development life cycle (SDLC) model selected for Secure-BMS, its application to the project, the proposed development schedule, and the principal risks associated with implementation. It also presents an indicative cost and effort estimate for the prototype.

3.1 SDLC Model Chosen and Justification

3.1.1 Selected Software Development Life Cycle Model

The **Agile development model with an incremental and iterative approach** is selected as the most suitable process model for Secure-BMS. The project can be divided into manageable functional increments, with each increment implemented and tested before further functionality is added.

Agile development emphasises iterative progress, continuous feedback, adaptation to changing requirements, and the delivery of working increments. These characteristics are useful for Secure-BMS because hardware integration and software integration can reveal problems that are not apparent during initial design.

For example, the thermistor measurement circuit had to be exercised before the temperature-processing logic could be evaluated meaningfully. Similarly, MQTT message handling, AES-GCM authentication, database insertion, and replay-state tracking required separate development and testing activities before they could be integrated into the overall data-processing path.

An incremental development approach allows the project to address such issues without requiring the entire system to be completed before testing begins.

The general development cycle adopted for Secure-BMS is:

1. **Requirement identification:** identify the intended functions, constraints, hardware interfaces, and software components.
2. **System design:** define the architecture, interfaces, data formats, and component responsibilities.
3. **Incremental implementation:** implement a small set of related functions at a time.
4. **Component testing:** verify each component against its expected behaviour.
5. **Integration:** connect tested components and verify the interactions between them.
6. **Evaluation and refinement:** examine observed behaviour, correct defects, and improve the implementation.
7. **Documentation:** record the design, implementation status, test evidence, limitations, and proposed enhancements.

The cycle is iterative rather than strictly linear. A test result may require the developer to revisit a previous design or implementation decision.

Figure 3.1: Iterative Agile development cycle



3.1.2 Application of Agile in Secure-BMS

Agile principles are applied by dividing Secure-BMS into functional increments. Each increment addresses a recognisable part of the system and provides a basis for testing before further integration.

The project has progressed through hardware experimentation, embedded firmware development, communication setup, secure message processing, and database integration. The following table describes the principal increments and their relationship to the current implementation.

Table 3.1: Incremental development plan for Secure-BMS

Increment	Main activities	Expected output	Status based on available evidence
Increment 1:	Identify system	System	Architecture and

Requirements and architecture	functions, hardware interfaces, telemetry requirements, and software components.	requirements and architecture.	implementation requirements established; formal approval records not established.
Increment 2: Hardware and sensing	Connect the ESP32, ADS1115, thermistor circuit, comparator signal, and buzzer interface.	Working sensing and local alert-related hardware.	Prototype components exercised; calibration and comprehensive safety validation remain incomplete.
Increment 3: Embedded firmware	Implement sensor acquisition, median filtering, temperature estimation, state evaluation, fault checks, and telemetry preparation.	Firmware for monitoring and reporting.	Implemented in the current firmware baseline.
Increment 4: Wireless communication	Configure Wi-Fi and MQTT telemetry publication and reception.	Device-to-backend communication path.	Implemented in the prototype.
Increment 5: Secure telemetry	Implement AES-128-GCM encryption, nonce handling, authenticated decryption, and input validation.	Authenticated telemetry-processing path.	Implemented.
Increment 6: Data storage and replay tracking	Store telemetry and implement persistent replay-state checks.	PostgreSQL telemetry and replay-state tables.	Implemented; isolated replay tests passed; live MQTT-path replay testing remains pending.
Increment 7: Dashboard and backend integration	Connect monitoring software to stored data and expose the dashboard.	Software-based monitoring interface.	Backend and Streamlit components are part of the prototype.
Increment 8: Testing and documentation	Consolidate evidence, test integration behaviour, record limitations, and prepare the report.	Test results and project documentation.	Documentation is in progress; further integration and validation work remains.
Increment 9: Advanced analytics	Prepare validated data and evaluate anomaly-detection algorithms.	Evaluated analytical extension.	Planned or requiring further validation; no validated model performance is claimed.

The table distinguishes completed implementation from work that requires additional evidence. This is important because a component can be implemented without having passed every possible integration or reliability test.

For example, the replay-state mechanism has passed isolated tests covering fresh and increasing counters, duplicate and stale counters, and session reuse. However, these tests do not establish that a replay attack has been successfully rejected through the complete live MQTT callback. The latter must remain a separate integration-test objective.

Similarly, the firmware includes a MOSFET gate interface, but the current `LOAD_ENABLE = false` configuration means that automated load switching should not be reported as an active demonstrated feature.

3.1.3 Justification for the Selected Model

The Agile model is suitable for Secure-BMS for the following reasons.

1. Hardware and software dependencies

The project integrates physical sensing hardware with firmware and software services. Testing the sensing hardware before developing higher-level monitoring functions helps reduce uncertainty. Incremental development supports this dependency by allowing the physical measurement path to be evaluated first.

2. Support for experimental development

Hardware experiments may reveal issues such as unstable readings, incorrect wiring, unsuitable assumptions about a sensor, or unexpected communication behaviour. An iterative process allows the implementation to be refined as evidence becomes available.

3. Modular implementation

The system can be divided into embedded sensing, state evaluation, telemetry communication, secure processing, storage, backend, and dashboard modules. Developing and testing these modules separately makes defects easier to isolate.

4. Early testing

Component-level testing can begin before the entire system is complete. For example, the temperature measurement path can be exercised independently of the dashboard, while replay-state logic can be tested separately from live MQTT communication.

5. Adaptability

The project includes functions whose validation requirements may evolve during development. An iterative process makes it possible to prioritise core monitoring and communication functions while leaving advanced anomaly detection and broader validation for later increments.

6. Incremental risk reduction

Testing each increment reduces the likelihood that a problem in one layer remains undetected until final integration. It does not eliminate integration risk, but it provides opportunities to identify and address issues earlier.

7. Documentation of implementation status

An iterative process supports the maintenance of test evidence and defect records alongside implementation. This is particularly useful for Secure-BMS because the report must distinguish implemented functions, isolated test results, pending integration tests, and future enhancements.

Table 3.2 compares the selected approach with other common SDLC models.

Table 3.2: Comparison of SDLC models

Criterion	Waterfall	Agile / iterative	Prototyping
Development structure	Sequential phases.	Repeated development and feedback cycles.	Early construction of a prototype to clarify requirements.
Handling changing requirements	Changes may require revisiting completed phases.	Changes can be incorporated into subsequent iterations.	Requirements can be refined using prototype observations.
Early component testing	Depends on the project plan; formal testing is often scheduled later.	Testing can occur during each increment.	Early prototype testing is central to the approach.
Suitability for hardware experimentation	Possible, but less flexible when assumptions change frequently.	Suitable when hardware and software are developed and tested incrementally.	Useful for investigating uncertain hardware or interface requirements.
Main limitation	Changes late in development can be costly.	Requires clear priorities and disciplined tracking to prevent uncontrolled scope growth.	Prototype shortcuts may be mistaken for production-ready design.
Relevance to Secure-BMS	Could provide a formal phase structure but may be less adaptable to experimental findings.	Selected for the overall iterative development process.	Useful as a complementary technique for hardware and interface experiments.

Agile is selected as the overall process model because Secure-BMS requires iterative integration and refinement. Prototyping is used as a complementary development technique for exploring hardware behaviour and interfaces. The project should not be described as having followed a formally managed Scrum process unless sprint backlogs, assigned roles, sprint reviews, and related records were actually maintained.

3.2 Project Schedule (Gantt Chart / Sprint Plan)

Project scheduling helps organise activities, identify dependencies, allocate effort, and determine when individual components should be tested. For Secure-BMS, the schedule must account for both hardware and software activities because the embedded sensing path is a prerequisite for meaningful end-to-end monitoring.

The project schedule is organised into phases that correspond to the development increments described in Section 3.1. The schedule below is an indicative plan for reporting and project management. It is not a verified historical record of the exact dates on which each task was performed.

3.2.1 Development Schedule

The proposed schedule divides the work into nine phases. Each phase has a defined focus and a measurable output.

Phase 1: Requirements and architecture

The first phase identifies the problem, project objectives, hardware constraints, system interfaces, telemetry fields, security requirements, and intended software components. The principal output is a requirements and architecture baseline against which subsequent implementation can be evaluated.

Phase 2: Hardware setup and sensor validation

The second phase establishes the physical sensing arrangement using the ESP32, ADS1115, thermistor, comparator signal, buzzer, and MOSFET interface. Basic component behaviour is checked before it is incorporated into the full firmware.

Phase 3: Embedded firmware development

The third phase implements measurement acquisition, filtering, temperature estimation, fault checks, operating-state evaluation, output handling, and telemetry preparation. The resulting firmware forms the embedded monitoring layer.

Phase 4: Wireless communication

The fourth phase configures Wi-Fi connectivity and MQTT publication. The communication path is tested independently before secure telemetry processing and database integration are treated as complete.

Phase 5: Secure telemetry implementation

The fifth phase implements authenticated encryption and receiving-side decryption. It includes nonce generation and handling, topic-based authenticated data, envelope validation, and rejection of messages that fail authentication or input checks.

Phase 6: Backend and database integration

The sixth phase connects MQTT message reception to payload validation and PostgreSQL storage. It also implements persistent replay-state checks and isolates replay tests from the production telemetry database where appropriate.

Phase 7: Dashboard integration

The seventh phase connects the software monitoring interface to the data available from the backend and database. The main output is a dashboard that can be used to inspect the prototype's monitoring information.

Phase 8: Testing and report preparation

The eighth phase consolidates test cases, captures evidence, identifies outstanding defects, documents limitations, and prepares the final report. Integration testing includes the verification of message rejection and database behaviour.

Phase 9: Advanced analytics and future work

The final planned phase evaluates whether the collected data is suitable for anomaly-detection experiments. It may include data preparation, feature selection, model development, and evaluation. This phase is a planned extension and should not be marked as completed until the relevant work and results are documented.

Table 3.3: Proposed project schedule

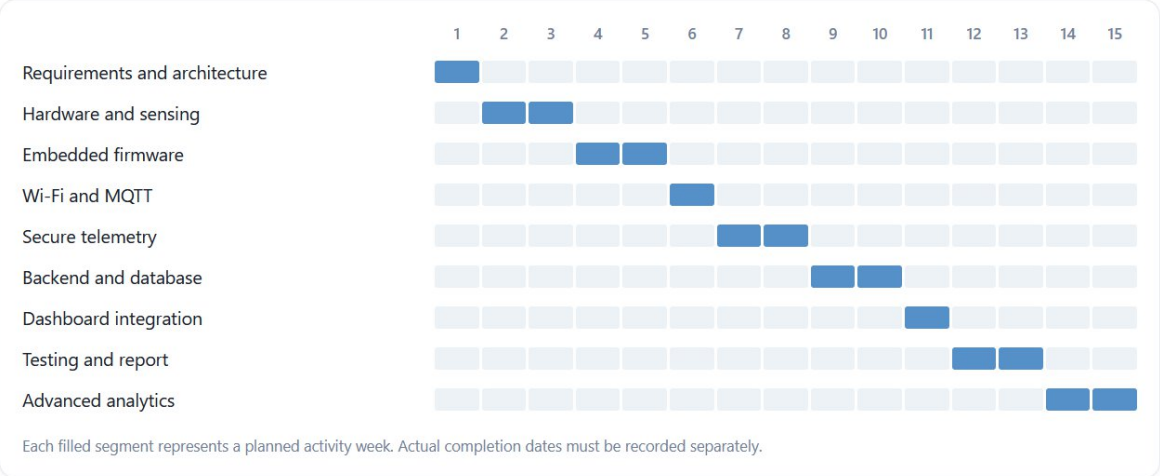
Phase	Activity	Indicative duration	Dependency	Deliverable
1	Requirements and architecture	1 week	None	Requirements and architecture baseline
2	Hardware setup and sensor validation	2 weeks	Phase 1	Exercised sensing and alert-related interfaces
3	Embedded firmware	2 weeks	Phase 2	Monitoring firmware
4	Wi-Fi and MQTT communication	1 week	Phase 3	Telemetry communication path
5	Authenticated encryption and validation	2 weeks	Phase 4	Secure telemetry processing
6	Backend, PostgreSQL and replay tracking	2 weeks	Phases 4–5	Persistent telemetry and replay-state handling
7	Dashboard integration	1 week	Phase 6	Monitoring dashboard
8	System testing and report preparation	2 weeks	Phases 2–7	Test evidence and report
9	Advanced analytics exploration	2 weeks	Phase 6; validated data required	Evaluated anomaly-detection extension, if completed

The durations are estimates for a planning baseline. They must not be interpreted as verified effort or actual completion dates. Several activities may overlap in a real project, while hardware faults or integration issues may extend the schedule.

3.2.2 Gantt Chart

A Gantt chart represents the expected timing and overlap of project activities. The following chart illustrates the proposed 15-week schedule. Week numbers are planning units rather than calendar dates.

Figure 3.2: Indicative Gantt chart for Secure-BMS



The schedule illustrates the intended order of development. Hardware and sensing precede embedded firmware, while secure telemetry follows the establishment of basic MQTT communication. Backend and database integration follows the availability of the communication and security components. Dashboard integration is scheduled after the data-processing path is available.

Testing and report preparation overlap with the final stages of development because documentation and verification should not be postponed entirely until the end of the project. Advanced analytics is scheduled as a later activity because it depends on the availability and quality of stored telemetry.

The Gantt chart is a proposed planning representation. It does not establish that each activity occurred during the corresponding week, and the final report should use actual dates if a verified project log is available.

3.3 Risk Analysis

Risk analysis identifies events that could prevent the project from meeting its objectives, assesses their potential effects, and defines actions to reduce their likelihood or impact. Secure-BMS contains both hardware-related and software-related risks because errors in the physical sensing path can affect the correctness of the software, while communication and database faults can affect the availability and integrity of monitoring information.

Risk management is particularly important for a prototype involving a lithium-ion cell. A software alert or monitoring dashboard must not be assumed to provide complete physical protection. Hardware experiments must use suitable precautions and must not depend solely on unvalidated software to prevent unsafe battery conditions.

3.3.1 Risk Identification and Mitigation

The following risks are relevant to the design, implementation, integration, and evaluation of Secure-BMS.

R1: Incorrect or unstable sensor readings

A thermistor circuit may produce inaccurate or unstable readings because of component tolerances, wiring problems, ADC limitations, or incorrect conversion assumptions. Such errors can affect temperature estimates and alert-state decisions.

Mitigation: inspect the sensing circuit, verify the ADC communication, use the implemented median filter, check for invalid readings, and compare temperature estimates against a suitable reference instrument during future calibration.

R2: Hardware connection errors

Incorrect wiring, poor contacts, or an incorrectly connected comparator or MOSFET may cause a component to behave unexpectedly. Hardware faults can also create misleading software observations.

Mitigation: verify connections before powering the circuit, test modules individually, document the GPIO assignments, and avoid changing multiple connections simultaneously during troubleshooting.

R3: Battery overheating or unsafe discharge

A lithium-ion cell may be damaged by unsuitable loading, excessive temperature, short circuits, or improper charging and discharging conditions. A prototype must not be assumed to provide complete protection simply because it includes a temperature sensor and buzzer.

Mitigation: use appropriate electrical and thermal precautions, apply a suitable controlled load, supervise experiments, and avoid relying exclusively on the prototype for safety-critical protection. Validate protective hardware independently before claiming reliable automatic disconnection.

R4: Network or MQTT broker failure

Wi-Fi disconnection, broker unavailability, or incorrect network configuration may interrupt telemetry delivery. A working local sensing subsystem does not guarantee that the remote software receives every message.

Mitigation: implement and test reconnection behaviour, check broker availability, inspect connection logs, and distinguish locally generated state changes from successfully received telemetry.

R5: Telemetry tampering or authentication failure

An encrypted message may be modified, malformed, or supplied with an invalid authentication tag. If the receiver processes unverified data, incorrect information may enter the monitoring workflow.

Mitigation: authenticate and decrypt messages before accepting their contents, validate decoded fields, reject failed authentication, and record security-relevant failures for debugging and testing.

R6: Replay of previously accepted telemetry

A previously valid message may be transmitted again. If the backend treats it as a new observation, the monitoring history may contain misleading duplicate or stale information.

Mitigation: maintain persistent session and counter state, reject repeated or stale sequences according to the implemented rules, and test duplicate, stale, and previous-session cases. Complete live MQTT-path replay testing remains necessary.

R7: Database failure or inconsistent state

Database connection errors, failed transactions, or inconsistent updates may prevent telemetry from being stored correctly. Replay-state updates must also remain consistent with the corresponding telemetry transaction.

Mitigation: use transactional database operations, validate records before insertion, handle connection errors, and test database failure and rollback scenarios. Additional recovery and backup testing is required before production deployment.

R8: Software integration defects

The firmware, MQTT message format, backend decoder, database schema, and dashboard may interpret fields differently. Such inconsistencies can prevent successful processing or cause incorrect presentation.

Mitigation: define a consistent telemetry schema, test components independently, validate interfaces during integration, and preserve representative test messages for regression testing.

R9: Incomplete anomaly-detection implementation

Machine-learning-based anomaly detection may be delayed by insufficient representative data, missing labels, or unsuitable model configuration. Implementing an algorithm without evaluating it may lead to unreliable conclusions.

Mitigation: treat machine learning as a separate development increment, validate the input data, establish evaluation criteria, and avoid claiming model performance before the required experiments are completed.

R10: Incomplete test coverage or documentation

A component may work under a limited test but fail under a different operating condition. Similarly, a report may overstate results if it does not distinguish component testing from integration testing.

Mitigation: maintain test cases and supporting evidence, record outstanding defects, identify the scope of each test, and explicitly document limitations and pending work.

Table 3.4: Risk identification and mitigation

Risk ID	Risk	Potential impact	Mitigation
R1	Unstable or inaccurate temperature readings	Incorrect temperature estimates or state transitions	Circuit checks, filtering, fault checks and reference-based calibration
R2	Hardware wiring or interface error	Incorrect component behaviour or hardware damage	Verify connections and test components individually
R3	Unsafe battery temperature or loading	Cell damage or hazardous conditions	Controlled experiments, supervision and independent safety precautions
R4	Wi-Fi or MQTT failure	Missing or delayed telemetry	Reconnection handling, broker checks and communication tests
R5	Invalid or modified encrypted message	Untrusted telemetry could be processed	AES-GCM authentication, payload validation and rejection of failures
R6	Replayed or stale telemetry	Misleading or duplicated monitoring records	Persistent replay-state tracking and integration testing
R7	Database or transaction failure	Missing records or inconsistent replay state	Transactions, exception handling, rollback and recovery tests
R8	Firmware/backend schema mismatch	Rejected messages or incorrect interpretation	Consistent schema, interface validation and regression testing
R9	Incomplete ML implementation	Unsupported anomaly-detection claims	Separate development phase, suitable data and model evaluation
R10	Inadequate testing or reporting	Undetected defects or overstated results	Test evidence, defect records and explicit implementation-status labels

3.3.2 Risk Assessment

A qualitative risk assessment can be used to prioritise the risks that require the greatest attention. For this report, each risk is assigned a likelihood score and an impact score on a five-point scale.

- **Likelihood:** 1 = Rare, 2 = Unlikely, 3 = Possible, 4 = Likely, 5 = Very likely.

- **Impact:** 1 = Negligible, 2 = Minor, 3 = Moderate, 4 = Major, 5 = Severe.
- **Risk score:** likelihood multiplied by impact.

The score provides a prioritisation aid rather than a statistically measured probability. The values below are preliminary engineering judgements and should be revised if the project has documented failure rates, test results, or a formal risk-assessment record.

Table 3.5: Qualitative risk assessment

Risk ID	Likelihood (1–5)	Impact (1–5)	Score	Priority
R1: Sensor inaccuracies	3	4	12	High
R2: Hardware connection errors	3	4	12	High
R3: Unsafe battery operation	2	5	10	High
R4: Network or broker failure	3	3	9	Medium
R5: Telemetry authentication failure	2	5	10	High
R6: Replay of stale telemetry	3	4	12	High
R7: Database or transaction failure	2	4	8	Medium
R8: Software integration defects	3	4	12	High
R9: Incomplete anomaly detection	4	2	8	Medium
R10: Inadequate testing or documentation	3	4	12	High

Note: These scores are indicative planning estimates, not measurements derived from a completed formal risk assessment.

The highest-priority risks concern sensor reliability, hardware correctness, replay handling, software integration, and test evidence. These risks can directly affect whether the monitoring information is trustworthy and whether the system behaves as expected.

Battery safety deserves special treatment even when the assigned likelihood is relatively low. Its impact may be severe, so appropriate physical precautions and independent protective measures remain necessary throughout development and testing.

Replay protection also requires more than an isolated logic test. Although the backend replay-state mechanism has passed isolated tests, the complete message path must be exercised to verify that a repeated message received through the live MQTT callback is rejected and does not create an additional telemetry record.

Similarly, the anomaly-detection risk is primarily a project-completion and evidence risk at this stage. The appropriate response is to keep it as a separate planned increment and avoid treating it as a completed safety function without verified implementation and evaluation.

3.4 Cost and Effort Estimation (Optional)

Cost and effort estimation provides an indicative view of the resources required to develop the prototype. Secure-BMS involves both physical components and software development activities. Its total cost therefore includes hardware procurement, supporting equipment, software development effort, testing, and documentation.

The project uses an ESP32 development board, an ADS1115 ADC module, a thermistor circuit, comparator and buzzer components, a MOSFET interface, a physical 18650 cell, and associated wiring and prototyping materials. The software stack includes embedded development tools, MQTT communication software, Python-based backend components, PostgreSQL, and Streamlit.

Because verified purchase invoices, a complete procurement record, and a measured time log are not established in this chapter, the following estimate is a planning template rather than a claim about the project's actual expenditure.

3.4.1 Hardware Cost Estimation

The hardware budget should be calculated using actual purchase prices and quantities. The following table identifies the main cost categories.

Table 3.6: Hardware cost estimation template

Component or category	Quantity	Unit price (₹)	Subtotal (₹)
ESP32 development board	1	To be filled from invoice	Quantity × unit price
ADS1115 ADC module	1	To be filled from invoice	Quantity × unit price
NTC thermistor and divider components	As purchased	To be filled from invoice	Quantity × unit price
LM339 comparator and associated components	As purchased	To be filled from invoice	Quantity × unit price
Active buzzer	1	To be filled from invoice	Quantity × unit price
IRLZ44N MOSFET and associated components	As purchased	To be filled from invoice	Quantity × unit price
18650 lithium-ion cell	1	To be filled from invoice	Quantity × unit price
Charger and protection modules	As purchased	To be filled from invoice	Quantity × unit price
Breadboard, prototyping board and wiring	As purchased	To be filled from invoice	Quantity × unit price
Miscellaneous components	As purchased	To be filled from invoice	Quantity × unit price
Total hardware cost			**Sum of verified subtotals**

The table should be completed using the actual procurement records. No total should be reported until the quantities and purchase prices have been verified. Components already owned before the project began may be listed separately as existing resources rather than treated as new expenditure.

3.4.2 Software Cost Estimation

The project uses several software tools and frameworks. Some may be available without a licence fee, depending on the edition, usage, and applicable terms. Development effort nevertheless represents a resource cost even when the software itself is free to use.

Table 3.7: Software and development-resource estimation

Resource	Purpose	Cost treatment
Arduino IDE	ESP32 firmware development	Record any applicable actual cost; do not assume a licence fee without verification
Visual Studio Code	Backend and software development	Record any applicable actual cost
Mosquitto MQTT broker	MQTT messaging	Record deployment or hosting costs, if any
PostgreSQL	Persistent data storage	Record deployment or hosting costs, if any
Python, FastAPI and Streamlit	Backend and dashboard development	Record any applicable hosting or service costs
Git and GitHub	Version control and repository hosting	Record the actual plan or service costs, if applicable
Development computer and internet access	Software development and connectivity	Allocate costs only if the project accounting method requires them
Developer effort	Implementation, testing and documentation	Estimate using recorded hours and an agreed hourly rate

The use of an open-source or no-cost software component does not mean that the total project cost is zero. Hardware, development time, test equipment, and operational resources must also be considered.

3.4.3 Effort Estimation

The project effort can be estimated by recording the number of hours spent on each major activity. If the project is evaluated as an academic exercise, these figures may be used to understand the distribution of work rather than to calculate a commercial project price.

Table 3.8: Development effort estimation template

Activity	Estimated effort (hours)
Requirements and architecture	To be filled using the project log
Hardware assembly and sensing tests	To be filled using the project log
Embedded firmware development	To be filled using the project log
MQTT and network configuration	To be filled using the project log
Encryption and secure message processing	To be filled using the project log
Backend and PostgreSQL integration	To be filled using the project log
Dashboard development	To be filled using the project log
Testing and defect correction	To be filled using the project log
Report preparation and evidence collection	To be filled using the project log
Total development effort	**Sum of recorded hours**

If a monetary estimate of developer effort is required, it can be calculated using:

Estimated labour cost = Total development hours × Hourly rate

The hourly rate should be selected according to the intended estimation method. For an academic prototype, it may be more appropriate to report the hours spent rather than assign a commercial labour rate without an agreed basis.

3.4.4 Cost and Effort Limitations

The estimates in this section are intentionally left as templates where verified figures are unavailable. The actual cost depends on the purchased component variants, quantities, shipping charges, existing equipment, and any software or hosting services used.

The development-effort estimate also depends on whether troubleshooting, repeated hardware experiments, integration debugging, and report preparation are included. A reliable final estimate should use a consistent accounting method and distinguish actual recorded effort from planned effort.

In conclusion, project planning for Secure-BMS is based on an iterative development approach that supports component-level testing and gradual integration. The proposed schedule organises the work into hardware, firmware, communication, security, backend, dashboard, and evaluation phases. Risk analysis identifies sensor reliability, hardware safety, replay handling, and integration testing as important areas requiring attention. Cost and effort estimation should be finalised using verified procurement and time records rather than unsupported figures.

4. Requirements Analysis

4.1 Functional Requirements

Functional requirements define the essential operations and services that the Secure-BMS system must provide. The system is designed to monitor a physical 18650 lithium-ion battery cell, identify abnormal operating conditions, generate local alerts, transmit telemetry securely, store validated data, and provide monitoring information through a dashboard.

FR-01: Battery Temperature Monitoring

The system shall monitor the temperature associated with the battery cell using an NTC thermistor and an ADS1115 analog-to-digital converter. The thermistor forms a voltage-divider circuit with a 10 kΩ resistor. The ESP32 acquires the corresponding digital measurements and estimates temperature using the thermistor parameters. A nine-sample median-filtering method is incorporated to reduce the influence of isolated measurement fluctuations.

FR-02: Temperature Alert Detection

The system shall identify configured temperature-related alert conditions. An LM339 comparator provides a digital signal to GPIO26 of the ESP32, while a potentiometer allows adjustment of the comparator threshold. The firmware processes the comparator input and associated measurement information to represent system conditions through states such as `NORMAL`, `TEMPERATURE_ALERT`, and `SENSOR_FAULT`.

FR-03: Audible Alert Generation

The system shall provide a local audible indication when the configured alert condition occurs. An active buzzer connected to GPIO25 is controlled using an active-low signal. This function enables the system to indicate abnormal conditions locally without depending entirely on remote monitoring.

FR-04: Sensor Fault Identification

The system shall identify supported sensor or measurement faults and represent them through the system-status logic. The firmware incorporates fault-checking mechanisms and a `SENSOR_FAULT` state. This requirement helps distinguish potentially invalid measurements from normal operating conditions. Comprehensive detection of all possible hardware and sensor failures requires additional validation.

FR-05: Telemetry Preparation

The system shall prepare telemetry messages containing supported measurement values, operating states, and relevant monitoring statistics. The firmware maintains information such as peak temperature, minimum and maximum voltage fields, alert count, alert duration, and sensor-fault count. The validity of each physical measurement depends on the corresponding sensor path being properly connected and calibrated.

FR-06: Secure Telemetry Communication

The system shall protect telemetry during transmission using authenticated encryption. The current implementation employs AES-128-GCM and an MQTT-based communication mechanism. The encrypted message envelope contains the encryption identifier, nonce, and ciphertext. The MQTT topic `secure-bms/telemetry` is used for telemetry publication, and the topic is incorporated as additional authenticated data (AAD).

FR-07: Telemetry Decryption and Validation

The backend shall authenticate and decrypt incoming telemetry before processing it. The `decode_telemetry_payload(payload, topic)` function validates the encrypted envelope and verifies the authentication tag. Messages that fail authentication or payload validation shall be rejected rather than processed as valid telemetry.

FR-08: Replay Protection

The backend shall reject duplicate, stale, or otherwise disallowed previously received telemetry messages according to the implemented replay-protection rules. The nonce contains a session identifier and message counter. PostgreSQL tables maintain replay-related state,

and the backend checks the received nonce before accepting the corresponding telemetry record.

Unit-level and database-level replay tests have been performed for fresh messages, increasing counters, duplicate or stale counters, new sessions, previously seen sessions, and transaction rollback. End-to-end replay testing through the live MQTT message callback remains necessary.

FR-09: Telemetry Storage

The system shall store accepted telemetry in a PostgreSQL database. The Python backend receives MQTT messages, validates the payload, applies replay checks, and stores accepted records. Database transactions are used to maintain consistency between replay-state updates and telemetry insertion.

FR-10: Dashboard Monitoring

The system shall provide a dashboard for viewing collected telemetry and relevant system-status information. Streamlit is used for the dashboard, while the backend and database support data processing and persistence. Complete dashboard functionality depends on the successful operation of the connected broker, backend, and database services.

FR-11: Anomaly Detection

The system is intended to support the identification of unusual battery-monitoring patterns using machine-learning techniques. Isolation Forest and Random Forest have been considered for this purpose. The operational accuracy and effectiveness of the anomaly-detection component require further testing and validation before it can be considered a verified system capability.

4.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that influence the reliability, security, performance, maintainability, and usability of the Secure-BMS system.

NFR-01: Security

The system shall protect telemetry against unauthorized disclosure and modification during communication. AES-128-GCM provides confidentiality and authentication, while backend validation and replay-protection logic provide additional security controls. Secure key management, correct nonce handling, and appropriate network configuration are necessary for maintaining these protections.

NFR-02: Reliability

The system should maintain consistent monitoring and communication during normal experimental operation. Wi-Fi reconnection logic and filtered sensor measurements are implemented in the firmware. Database transactions support consistent telemetry storage and

replay-state management. Recovery from all possible hardware, network, and database failures requires further testing.

NFR-03: Measurement Consistency

The system shall use consistent measurement-processing methods and status interpretation. Median filtering helps reduce isolated measurement fluctuations, while temperature estimation depends on the thermistor parameters and voltage-divider configuration. Measurements that have not been calibrated or validated shall not be treated as verified physical values.

NFR-04: Performance

The system should process sensor measurements, evaluate supported alert conditions, and transmit telemetry with sufficiently low delay for experimental monitoring. Performance depends on the sampling interval, filtering operation, cryptographic processing, network connectivity, broker performance, and database operations. The configured sampling procedure does not establish a verified end-to-end response-time guarantee.

NFR-05: Availability

The system should resume communication following supported network interruptions. The ESP32 firmware includes Wi-Fi reconnection logic. However, complete remote monitoring depends on the availability of the hardware, wireless network, MQTT broker, backend, database, and dashboard.

NFR-06: Maintainability

The system shall be organized into components that can be developed, tested, and modified independently wherever practical. The project separates embedded firmware, communication, backend processing, database operations, dashboard presentation, and anomaly-detection logic. Version control and automated testing can support maintainability and regression detection.

NFR-07: Usability

The system should present monitoring information in a form that can be understood without requiring users to inspect low-level hardware signals or database records directly. The dashboard is intended to display telemetry and system status, while the buzzer provides local audible indication of configured alerts.

NFR-08: Scalability

The architecture should permit future extensions involving additional sensors, larger telemetry histories, and more comprehensive analytical functions. The scalability of the current implementation must be established through workload testing before any specific device capacity or message-rate guarantee can be claimed.

NFR-09: Safety

The system should assist in identifying and communicating potentially abnormal battery conditions. The current prototype provides temperature monitoring and local alert functionality; however, automated load switching is not active in the present firmware configuration. The prototype shall not be treated as a certified battery protection system or as a replacement for appropriate electrical and laboratory safety measures.

NFR-10: Testability

The software shall support repeatable testing of sensor processing, state transitions, encryption, message authentication, replay protection, database consistency, and dashboard integration. Existing unit-level and database-level replay tests provide partial verification. Additional end-to-end and hardware-in-the-loop testing is required to establish overall system performance.

4.3 Software Requirements Specification (SRS) Summary

4.3.1 Purpose

The purpose of the Secure-BMS system is to develop a cyber-physical testbed for monitoring a physical 18650 lithium-ion battery cell and securely communicating its telemetry to a software monitoring platform. The system integrates embedded sensing, local alert logic, encrypted communication, backend validation, persistent data storage, and dashboard presentation.

The project demonstrates the integration of physical measurements and software-based monitoring within a security-conscious cyber-physical architecture. It also provides a foundation for future development of machine-learning-based anomaly detection and additional battery-monitoring capabilities.

4.3.2 System Overview

The Secure-BMS architecture consists of a physical sensing layer, an embedded processing layer, a communication layer, and a software monitoring layer.

The physical sensing layer comprises the battery cell, NTC thermistor, voltage-divider circuit, and ADS1115 analog-to-digital converter. The ESP32 processes sensor measurements, evaluates supported fault conditions, maintains event states, and prepares telemetry. The LM339 comparator and active buzzer provide the configured local alert functionality.

The communication layer uses Wi-Fi and MQTT to transmit AES-128-GCM-encrypted telemetry to the Mosquitto broker. The backend receives messages, authenticates and decrypts their contents, validates the payload, and applies replay-protection checks. Accepted telemetry is stored in PostgreSQL and made available to the Streamlit dashboard.

Machine-learning-based anomaly detection is a planned extension whose effectiveness requires further validation.

4.3.3 Inputs

The system receives physical measurements, hardware status signals, configuration parameters, and incoming communication messages. The principal inputs are presented in Table 4.1.

Table 4.1: System Inputs

Input	Source	Purpose
Thermistor voltage	NTC voltage-divider circuit through ADS1115	Temperature estimation
Comparator output	LM339 comparator connected to GPIO26	Threshold-condition indication
Temperature threshold	Comparator potentiometer	Configuration of the switching threshold
Network configuration	ESP32 firmware	Establishment of Wi-Fi connectivity
Encryption parameters	Embedded security implementation	Telemetry encryption and authentication
MQTT payload	Mosquitto broker	Delivery of encrypted telemetry to the backend
Replay-state records	PostgreSQL	Identification of stale or previously seen messages

The ADS1115 and thermistor circuit constitute the implemented temperature-measurement path. Other physical measurements must be independently validated before being treated as reliable system inputs.

4.3.4 Processing

The system processes inputs through a sequence of embedded measurement processing and software-based validation operations.

First, the ADS1115 acquires the voltage associated with the thermistor circuit. The ESP32 applies nine-sample median filtering and estimates temperature using the configured thermistor model. The firmware evaluates supported fault conditions and event states, processes the comparator input, and controls the local buzzer.

The firmware then prepares the telemetry message and encrypts it using AES-128-GCM. The encrypted payload is published through MQTT over Wi-Fi. The backend authenticates and decrypts the received message, validates its contents, and checks its nonce against the database-backed replay state. If the message passes these checks, the telemetry and corresponding replay-state updates are committed to PostgreSQL within a transaction.

Finally, the dashboard provides access to the collected monitoring information. Machine-learning-based anomaly analysis may be incorporated once the relevant model and its performance have been validated.

4.3.5 Outputs

The system generates local alerts, encrypted telemetry, validation results, stored records, and dashboard information.

Table 4.2: System Outputs

Output	Description
Estimated temperature	Temperature calculated from the thermistor measurement
Event state	Normal, temperature-alert, or sensor-fault state
Audible alert	Buzzer indication of a configured abnormal condition
Encrypted telemetry	MQTT payload containing the encryption identifier, nonce, and ciphertext
Validation result	Acceptance or rejection based on authentication and payload checks
Replay-protection result	Acceptance or rejection according to replay-state rules
Stored telemetry	Validated records persisted in PostgreSQL
Dashboard information	Monitoring measurements and system status
Diagnostic statistics	Alert counts, temperature statistics, and sensor-fault information maintained by the firmware

The accuracy and availability of these outputs depend on the validity of the measurements and the operation of the connected hardware and software components.

4.3.6 Assumptions and Constraints

Assumptions

1. The battery cell and associated sensing circuitry are connected according to the experimental setup.
2. The thermistor parameters and voltage-divider configuration are sufficiently characterized for temperature estimation.
3. The ESP32, ADS1115, comparator, and buzzer operate within their configured electrical limits.
4. Wi-Fi connectivity and the MQTT broker are available when remote telemetry is required.
5. The backend uses the appropriate encryption key and the correct MQTT topic for authentication.
6. PostgreSQL is available for telemetry storage and replay-state management.
7. The backend and dashboard services are operational when remote monitoring is required.

Constraints

1. Temperature-estimation accuracy depends on sensor characteristics, calibration, and electrical connections.
2. The current implementation is an experimental single-cell monitoring setup and does not establish suitability for arbitrary battery configurations.

3. Remote monitoring depends on network connectivity and the availability of the broker, backend, and database.
4. Encryption security depends on appropriate key protection and correct nonce management.
5. End-to-end replay testing through live MQTT reception remains incomplete.
6. INA219 measurements have not been fully validated and cannot be assumed to provide verified current-monitoring data.
7. Automated load switching is not active in the current firmware configuration.
8. The anomaly-detection component requires additional validation before its effectiveness can be established.
9. The system is not certified as a complete battery protection solution.

4.4 Hardware and Software Requirements

4.4.1 Hardware Requirements

The hardware components support battery sensing, embedded processing, local alert generation, and prototype development. The principal components are listed in Table 4.3.

Table 4.3: Hardware Requirements

Component	Purpose
ESP32 development board	Embedded processing, event logic, and Wi-Fi communication
18650 lithium-ion cell	Physical battery used for experimental monitoring
NTC thermistor	Temperature sensing
10 kΩ resistor	Thermistor voltage-divider circuit
ADS1115 ADC module	Conversion of analog sensor voltage into digital measurements
LM339 comparator	Hardware threshold detection
Potentiometer	Comparator threshold adjustment
Active buzzer module	Local audible alert
IRLZ44N MOSFET	Component included in the load-switching circuit
Breadboard and connecting wires	Prototype assembly and electrical interconnection
USB cable and power source	Development-time power and firmware uploading
Digital multimeter	Basic electrical measurements and troubleshooting

The available battery is labelled 3.7 V, 2000 mAh, and 7.4 Wh. The ADS1115 is detected at I²C address `0x48`, while the LM339 comparator output, buzzer control, and MOSFET gate are connected to GPIO26, GPIO25, and GPIO27, respectively.

The INA219 module is detected at I²C address `0x40`, but its measurements have not been fully validated. It is therefore treated as a component requiring further verification rather than a confirmed measurement source. Battery charging and protection components must be connected according to their manufacturers' documentation and suitable electrical safety practices.

4.4.2 Software Requirements

The software components support embedded programming, secure communication, backend processing, data storage, dashboard presentation, and verification. The principal requirements are listed in Table 4.4.

Table 4.4: Software Requirements

Software or technology	Purpose
Arduino IDE	ESP32 firmware development and uploading
ESP32 Arduino platform	Embedded development support
Embedded C++	Sensor processing, alert logic, encryption, and telemetry preparation
ADS1X15-compatible library	ADS1115 communication
Wi-Fi and MQTT libraries	Network connectivity and telemetry publishing
Mbed TLS	AES-128-GCM encryption and authentication
Mosquitto	MQTT message brokering
Python	Backend implementation and supporting software
FastAPI	Backend application interface
Paho MQTT client	MQTT message reception
PostgreSQL	Telemetry and replay-state storage
Psycopg	Python connectivity to PostgreSQL
Streamlit	Dashboard development
Base64 encoding and decoding	Representation of binary cryptographic fields
Pytest	Automated software testing

The embedded and backend software components must use compatible libraries and configuration settings. Encryption keys and other sensitive parameters must be kept separate from publicly shared source code. Development tools such as VS Code, Git/GitHub, StarUML, draw.io, Postman, and a project-tracking application may support implementation, testing, and documentation where applicable.

Chapter Summary

This chapter established the functional and non-functional requirements of Secure-BMS, summarized its software requirements specification, and identified the hardware and software components used in the prototype. The requirements cover temperature monitoring, local alerts, secure MQTT telemetry, backend validation, replay protection, data storage, and dashboard monitoring.

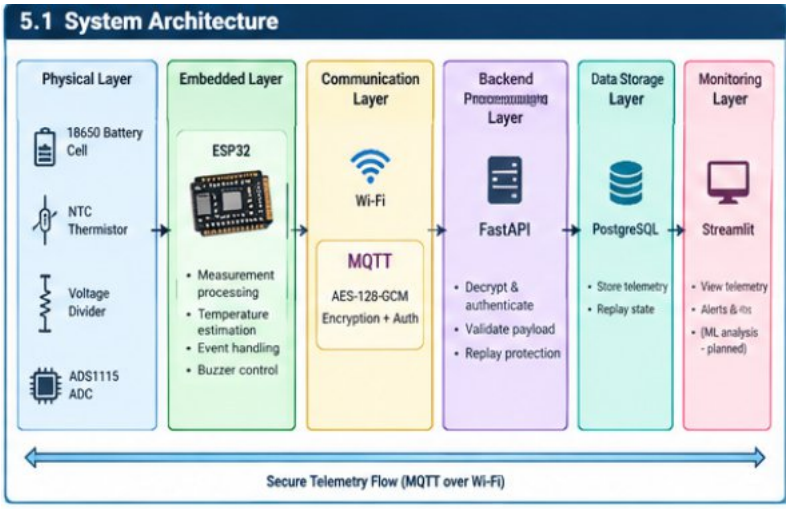
The analysis also identifies the current implementation boundaries, including unvalidated INA219 measurements, incomplete end-to-end replay testing, unverified anomaly-detection performance, and inactive automated load switching. These constraints provide the basis for the subsequent system-design and implementation chapters.

5. System Design

5.1 System Architecture

System architecture defines the organization of the Secure-BMS system and the interactions between its hardware and software components. The architecture is designed to integrate physical battery monitoring, embedded processing, secure telemetry communication, backend validation, persistent storage, and dashboard-based monitoring.

The system follows a layered cyber-physical architecture in which the ESP32 performs edge-level measurement processing and local alert handling, while the backend manages incoming telemetry, authentication, replay protection, and database operations. This separation supports modular development and allows individual components to be tested independently.



5.1.1 Overview

Secure-BMS is designed as a cyber-physical battery-monitoring testbed for a physical 18650 lithium-ion battery cell. The physical layer acquires sensor measurements, the embedded layer processes the measurements and generates local status information, and the communication layer transfers encrypted telemetry to the software backend.

The backend receives MQTT messages, decrypts and authenticates their contents, validates the payload, checks replay state, and stores accepted telemetry in PostgreSQL. A Streamlit dashboard provides the interface for viewing the collected monitoring information.

The architecture consists of the following major layers:

1. **Physical sensing layer:** Includes the lithium-ion battery cell, NTC thermistor, voltage-divider circuit, and ADS1115 analog-to-digital converter.
2. **Embedded processing layer:** Uses the ESP32 to acquire measurements, apply filtering, evaluate supported fault conditions, maintain event states, and control the local buzzer.
3. **Secure communication layer:** Uses Wi-Fi and MQTT to transfer telemetry protected by AES-128-GCM encryption and authentication.

4. **Backend processing layer:** Receives encrypted messages, verifies authentication, validates payloads, and applies replay-protection checks.
5. **Data storage layer:** Uses PostgreSQL to persist accepted telemetry and maintain replay-protection state.
6. **Monitoring and analysis layer:** Uses Streamlit to present monitoring information. Machine-learning-based anomaly analysis is an intended extension that requires further validation.

The architecture emphasizes the separation of local monitoring functions from remote software services. However, the current prototype should not be interpreted as providing a fully independent, certified battery-protection mechanism.

5.1.2 Architectural Components

The major architectural components and their responsibilities are described in Table 5.1.

Table 5.1: Architectural Components of Secure-BMS

Component	Responsibility
18650 lithium-ion cell	Provides the physical battery under observation
NTC thermistor	Senses temperature associated with the monitored cell
ADS1115 ADC	Converts the analog thermistor-circuit voltage into digital measurements
ESP32 controller	Processes measurements, evaluates supported fault conditions, and prepares telemetry
LM339 comparator	Produces a digital threshold-condition signal
Active buzzer	Provides local audible indication
AES-128-GCM module	Encrypts telemetry and authenticates message contents
Wi-Fi interface	Provides network connectivity for telemetry transmission
MQTT publisher	Publishes encrypted telemetry to the configured topic
Mosquitto broker	Receives and distributes MQTT messages
FastAPI backend	Hosts the backend application and message-processing functions
Paho MQTT client	Receives messages from the MQTT broker
Replay-protection module	Checks message counters and previously seen sessions
PostgreSQL database	Stores accepted telemetry and replay-state information
Streamlit dashboard	Presents collected monitoring information
Anomaly-detection component	Intended to identify unusual patterns after model validation

The ESP32 firmware uses the ADS1115 at I²C address `0x48`. The LM339 comparator output is connected to GPIO26, the active-low buzzer control uses GPIO25, and the IRLZ44N MOSFET gate is connected to GPIO27.

The firmware implements Wi-Fi reconnection, temperature estimation, median filtering, event-state processing, and encrypted telemetry preparation. The backend implements authenticated decryption and database-backed replay checks. Automated load switching is not active in the current firmware configuration, and the INA219 measurement path has not been fully validated.

5.1.3 System Data Flow

The system data flow describes how information travels from the physical battery-monitoring hardware to the dashboard.

Initially, the NTC thermistor and voltage-divider circuit generate a voltage corresponding to the temperature-dependent resistance. The ADS1115 samples this voltage and provides digital measurements to the ESP32. The firmware applies median filtering and estimates the corresponding temperature.

The ESP32 evaluates supported conditions and prepares a telemetry message containing measurement and status information. The message is encrypted using AES-128-GCM and published through MQTT over Wi-Fi.

The Mosquitto broker forwards the message to the backend subscriber. The backend verifies the encrypted envelope, authenticates and decrypts the message, validates the payload, and checks its nonce against the database-backed replay state. Accepted messages and replay-state updates are committed to PostgreSQL within a transaction.

Finally, the dashboard obtains monitoring information from the software data layer and presents it to the operator.

The principal data-flow sequence is:

Physical sensing → ESP32 processing → AES-128-GCM encryption → MQTT publication → Broker → Backend authentication and validation → Replay protection → PostgreSQL storage → Dashboard presentation.

If authentication or replay validation fails, the message is rejected rather than stored as accepted telemetry. A communication failure can interrupt remote monitoring even when local sensing remains operational.

5.2 Use Case Diagram

The use case model describes the interactions between external actors and the Secure-BMS system. It identifies the services available to the system operator and the supporting interactions with external software components.

The primary objective is to represent how an operator monitors battery information and how the system processes and stores telemetry.

Actors

The principal actors are described below.

1. System Operator

The system operator interacts with the monitoring interface to view battery measurements, inspect event states, and observe collected telemetry. The operator may also examine system diagnostics during development and testing.

2. Physical Battery and Sensor Assembly

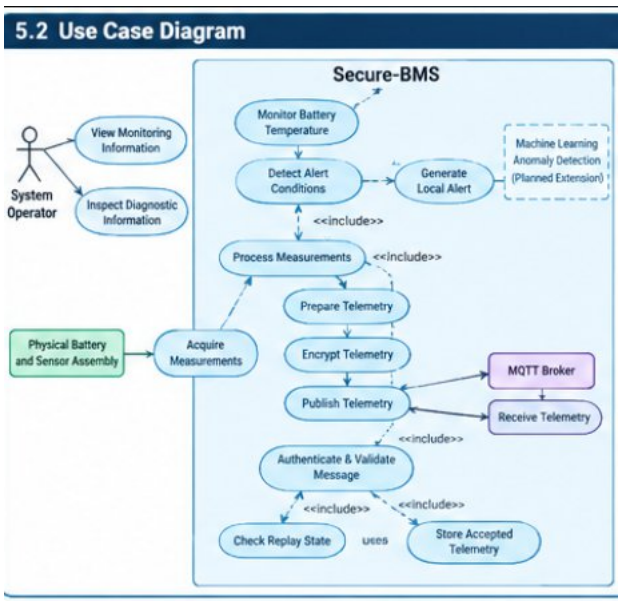
The battery and associated sensing components provide the physical conditions and measurement inputs used by the monitoring system. Although these components are not human users, they act as external physical entities in the broader system interaction model.

3. MQTT Broker

The MQTT broker acts as the communication intermediary between the ESP32 publisher and the backend subscriber. It receives published telemetry and forwards messages to the appropriate subscribers.

4. Backend and Database Services

The backend and database services receive and process telemetry, perform authentication and replay checks, and persist accepted records. These are supporting system components rather than independent human actors.



Principal Use Cases

The main use cases are listed in Table 5.2.

Table 5.2: Principal Use Cases

Use case	Description
Monitor battery temperature	Acquire and process thermistor measurements
Detect configured alert conditions	Evaluate supported threshold and fault conditions
Generate local alert	Activate the configured audible indication

Prepare telemetry	Assemble measurement and status information
Encrypt telemetry	Protect telemetry using AES-128-GCM
Publish telemetry	Transmit encrypted data through MQTT
Authenticate incoming telemetry	Verify message authenticity before processing
Validate payload	Check the structure and validity of decrypted data
Detect replay attempts	Reject messages that violate replay-state rules
Store accepted telemetry	Persist validated records in PostgreSQL
View monitoring information	Display collected telemetry through the dashboard
Inspect diagnostic information	Examine supported event and system-status information

The use case model establishes the intended responsibilities of the system. It does not imply that all proposed features, including validated machine-learning classifications or remote hardware control, are operational in the current prototype.

Use Case Diagram — diagram specification

The diagram should be drawn using StarUML or draw.io with a system boundary labelled *Secure-BMS*. Place the System Operator outside the boundary and connect the operator to *View Monitoring Information* and *Inspect Diagnostic Information*. Represent the battery and sensor assembly as an external input entity associated with *Acquire Measurements*. Connect the MQTT broker to *Publish Telemetry* and *Receive Telemetry*. Within the system boundary, include the use cases for *Process Measurements*, *Generate Local Alert*, *Encrypt Telemetry*, *Authenticate and Validate Message*, *Check Replay State*, and *Store Accepted Telemetry*.

The diagram should use associations for actor interactions and `<<include>>` relationships only where one use case necessarily invokes another. Machine-learning-based anomaly detection should be labelled as a planned extension if it is shown.

5.3 Class Diagram

The class diagram represents the principal software responsibilities and the relationships between the logical components of Secure-BMS. It provides a structural view of how measurement processing, telemetry security, message reception, persistence, and dashboard presentation can be organized.

The class model is a logical representation of the system architecture. It does not imply that every component is implemented as a separate object-oriented class in the current source code.

Main Components

1. SensorMonitor

The SensorMonitor component represents the responsibility for acquiring and processing measurements from the sensor interface.

Principal attributes include the ADC interface, thermistor configuration, and measurement status.

Principal operations include:

- `readSensor()`
- `filterSamples()`
- `calculateTemperature()`
- `checkSensorFault()`

2. BatteryMonitor

The BatteryMonitor component represents the embedded monitoring logic. It maintains the interpreted operating state and supported monitoring statistics.

Principal attributes include the current event state, peak temperature, and alert statistics.

Principal operations include:

- `evaluateCondition()`
- `updateEventState()`
- `updateStatistics()`

3. AlertController

The AlertController component represents the local indication logic associated with the comparator and buzzer.

Principal operations include:

- `readComparatorState()`
- `activateBuzzer()`
- `deactivateBuzzer()`

These operations describe the logical responsibilities of the alert component and do not establish an active automated load-disconnection function.

4. TelemetryManager

The TelemetryManager component prepares telemetry and coordinates its transmission.

Principal attributes include the message payload, MQTT topic, and message counter.

Principal operations include:

- `prepareTelemetry()`
- `encryptTelemetry()`
- `publishTelemetry()`

5. CryptoService

The CryptoService component represents the cryptographic functions used to protect telemetry.

Principal operations include:

- `encryptPayload()`
- `decryptPayload()`
- `verifyAuthenticationTag()`

The cryptographic implementation uses AES-128-GCM and the configured nonce and authenticated additional data.

6. MQTTMessageHandler

The MQTTMessageHandler component represents backend message reception and dispatch.

Principal operations include:

- `onMessage()`
- `decodeTelemetryPayload()`
- `validatePayload()`

The handler passes valid messages to replay validation and persistence operations.

7. ReplayProtection

The ReplayProtection component represents the logic that checks the nonce and the stored session and counter state.

Principal operations include:

- `validateNonce()`
- `checkSession()`
- `checkCounter()`
- `updateReplayState()`

The actual replay checks are implemented using database-backed state and transaction logic.

8. TelemetryRepository

The TelemetryRepository component represents database operations.

Principal operations include:

- `saveTelemetry()`
- `readTelemetry()`
- `updateReplayState()`

These operations describe the database responsibilities; the current implementation uses Python backend functions and PostgreSQL.

9. Dashboard

The Dashboard component represents the presentation layer for monitoring data.

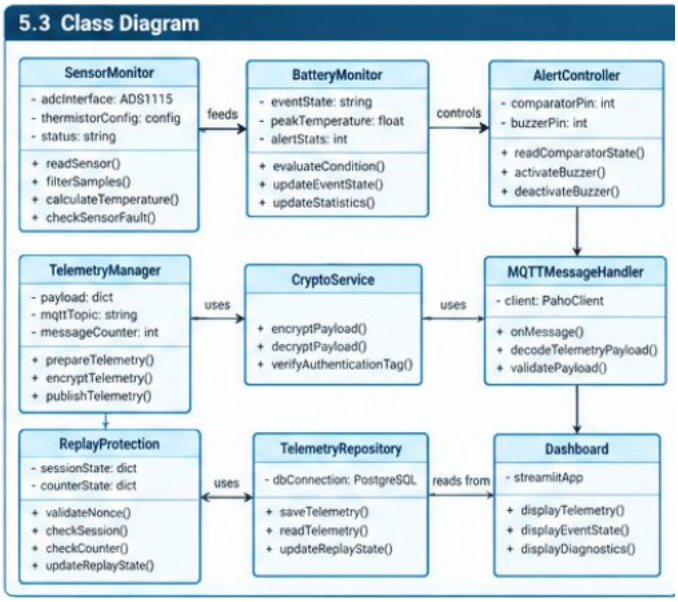
Principal operations include:

- `displayTelemetry()`
- `displayEventState()`
- `displayDiagnostics()`

The current dashboard is implemented using Streamlit.

Class relationships

SensorMonitor supplies measurements to BatteryMonitor. BatteryMonitor provides interpreted status information to TelemetryManager and AlertController. TelemetryManager uses CryptoService to protect messages before publication. On the receiving side, MQTTMessageHandler uses CryptoService to authenticate and decode incoming payloads, then invokes ReplayProtection and TelemetryRepository. Dashboard retrieves monitoring information from the data layer.



5.4 Sequence Diagram / Activity Diagram

Interaction diagrams describe the dynamic behaviour of Secure-BMS. The sequence diagram presents the order in which components exchange messages, while the activity diagram represents the workflow followed during measurement processing and telemetry acceptance.

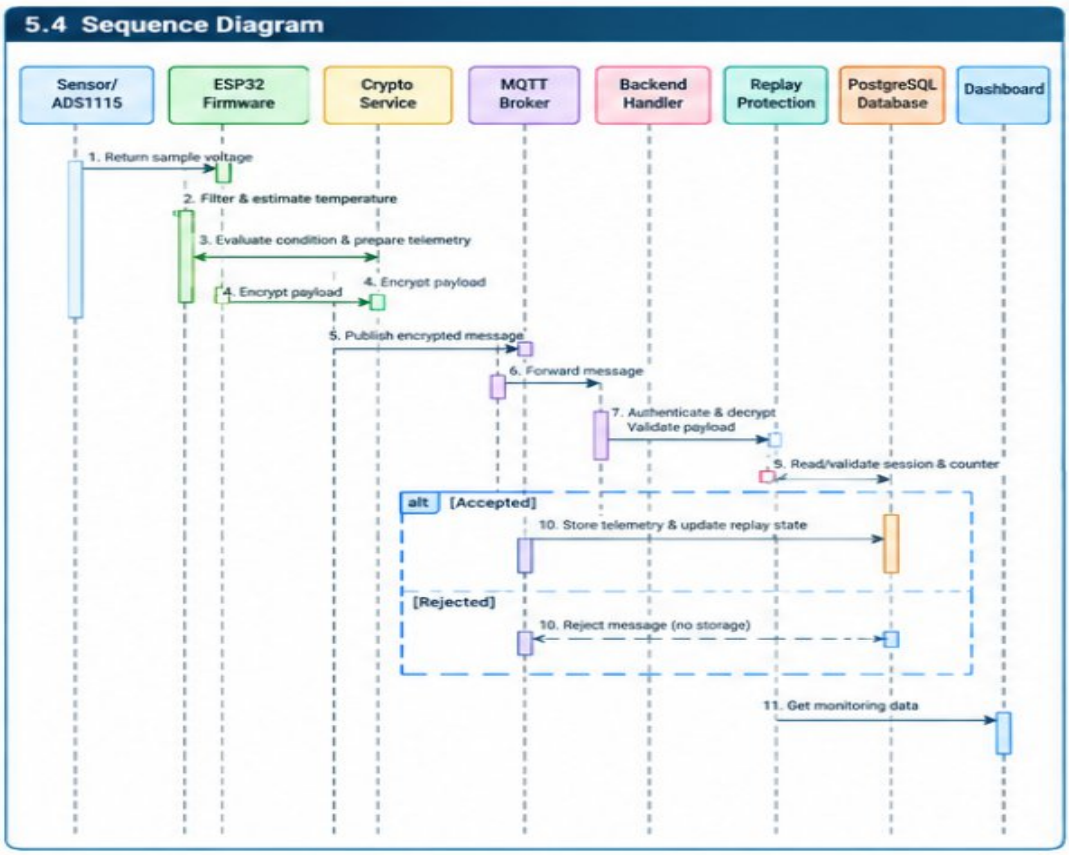
5.4.1 Sequence Diagram

The sequence diagram models the processing of one telemetry message from sensor acquisition to database storage.

The interaction begins when the ESP32 acquires a measurement from the ADS1115. The firmware filters the measurement, estimates temperature, evaluates supported conditions, and prepares the telemetry payload. The payload is encrypted and published to the MQTT broker.

The broker forwards the encrypted message to the backend subscriber. The backend authenticates and decrypts the payload, validates the resulting data, and checks the nonce against the replay state. If the message is accepted, the backend stores the telemetry and

updates the replay state in a database transaction. If validation or replay detection fails, the message is rejected.



Sequence Diagram — interaction order

The following participants should appear from left to right in the diagram:

1. Sensor/ADS1115
2. ESP32 Firmware
3. Crypto Service
4. MQTT Broker
5. Backend Message Handler
6. Replay Protection
7. PostgreSQL Database
8. Dashboard

The principal message sequence is:

1. Sensor/ADS1115 returns the sampled voltage.
2. ESP32 Firmware filters the measurement and calculates the temperature.
3. ESP32 Firmware evaluates the condition and prepares telemetry.
4. ESP32 Firmware requests encryption from Crypto Service.
5. The encrypted message is published to the MQTT Broker.
6. The MQTT Broker forwards the message to Backend Message Handler.
7. Backend Message Handler requests authenticated decryption and payload validation.

8. Backend Message Handler checks replay state through Replay Protection.
9. Replay Protection reads and validates the session and counter state in PostgreSQL.
10. If accepted, the backend stores telemetry and updates replay state within the transaction.
11. The Dashboard obtains the stored monitoring information for presentation.

An alternative path should show rejection when authentication, payload validation, or replay checks fail. In that path, the message must not be represented as successfully stored telemetry.

5.4.2 Activity Diagram

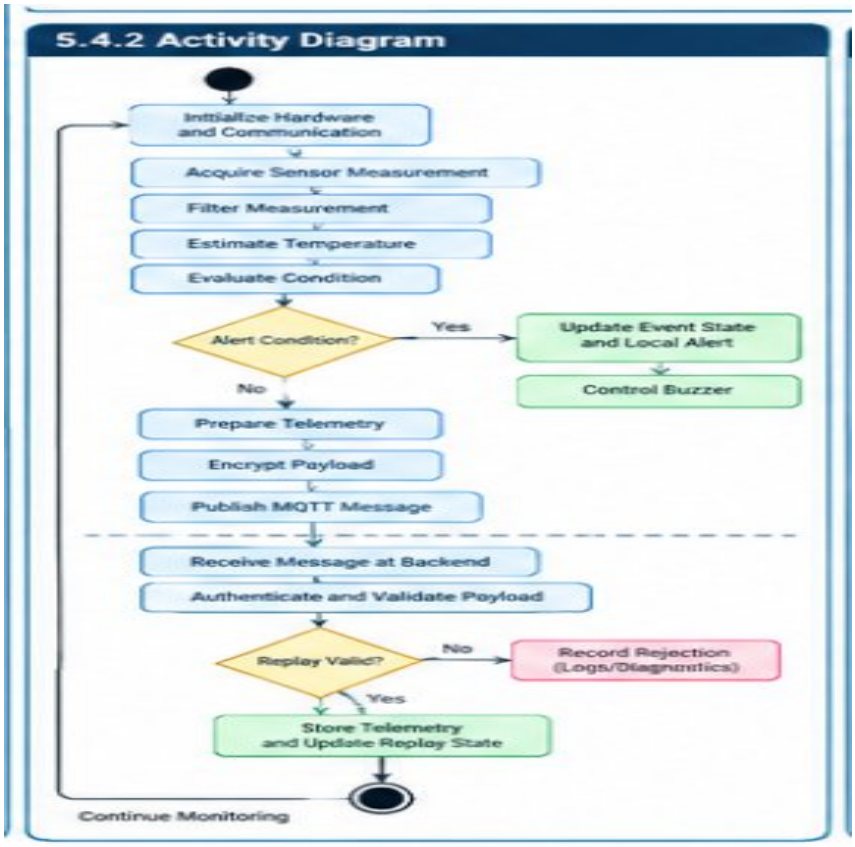
The activity diagram represents the control flow of the Secure-BMS monitoring and telemetry pipeline.

The process begins with system initialization and the establishment of the required hardware and communication interfaces. The ESP32 then repeatedly acquires sensor measurements, filters the readings, estimates temperature, and evaluates supported fault and alert conditions.

If a configured alert condition occurs, the local alert logic processes the condition and controls the buzzer as appropriate. The firmware then prepares and encrypts telemetry before attempting MQTT publication.

On the backend, received messages are authenticated and validated. A decision node determines whether the message passes cryptographic and payload checks. Valid messages proceed to replay-state validation. A second decision node determines whether the nonce satisfies the replay rules. Accepted messages are stored transactionally in PostgreSQL; rejected messages follow the error-handling path.

After processing, monitoring continues while the system remains operational.



Activity Diagram — principal activities

The diagram should include the following activities in sequence:

Start → Initialize Hardware and Communication → Acquire Sensor Measurement → Filter Measurement → Estimate Temperature → Evaluate Condition → Update Event State and Local Alert → Prepare Telemetry → Encrypt Payload → Publish MQTT Message → Receive Message at Backend → Authenticate and Validate Payload → Check Replay State.

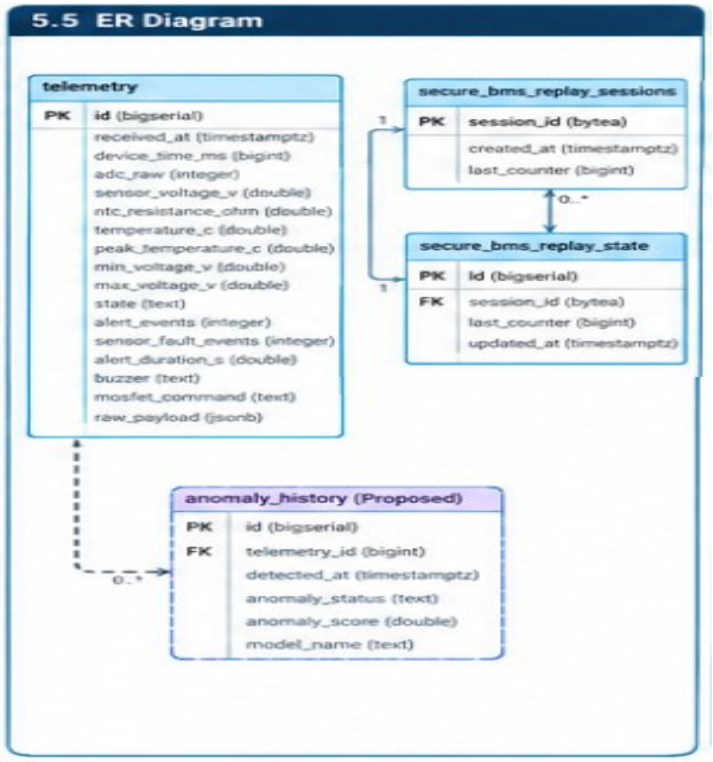
The replay decision branches into two paths. If the message is accepted, the system stores telemetry and updates replay state within a database transaction. If the message is rejected, the system records or handles the rejection according to the available diagnostic logic. Both paths return to the monitoring workflow where appropriate.

The diagram should not show automatic load disconnection as an active operation in the current configuration.

5.5 ER Diagram / Database Design

The database design describes how Secure-BMS organizes accepted telemetry and the information required for replay protection. PostgreSQL is used as the persistent storage system.

The database must support the insertion of accepted telemetry, the tracking of session and counter information, and consistent handling of duplicate or stale messages. Transactional operations ensure that replay-state updates and telemetry insertion are handled consistently.



5.5.1 Database Overview

The database layer stores monitoring records and replay-protection state. The backend validates incoming messages before attempting to persist them.

The current implementation uses the `secure_bms_replay_sessions` and `secure_bms_replay_state` tables to support replay detection. The telemetry storage structure contains accepted monitoring records, while replay-state records track the latest accepted counter for a session and information about previously seen sessions.

Database transactions are used so that the replay-state update and telemetry insertion are treated as one consistent operation. If an operation fails, the transaction can be rolled back to avoid accepting inconsistent replay state.

The ER model should distinguish between telemetry records and replay-management records. Their relationship should reflect the actual database schema and the implementation of the transaction logic.

5.5.2 Telemetry Entity

The Telemetry entity represents an accepted monitoring message stored by the backend.

The logical entity may contain the following attributes, subject to confirmation against the actual PostgreSQL schema:

Attribute	Description
Telemetry identifier	Identifies an individual stored record
Timestamp	Records when the telemetry was received or stored
Temperature	Stores the accepted temperature value, when present
Event state	Represents the reported system state
Payload data	Stores the validated measurement and status fields
Nonce	Identifies the cryptographic message nonce when stored by the implementation

The exact column names, data types, primary key, and nullability constraints must follow the implemented database definition. Fields should not be added to the ER diagram as though they exist in the database until the schema has been checked.

5.5.3 Anomaly History Entity

The Anomaly History entity represents the historical record of anomaly-related results when the anomaly-detection component is integrated with persistent storage.

A logical anomaly record may include an identifier, associated telemetry reference, detection timestamp, anomaly classification, and supporting result information. These attributes describe a proposed logical model and should only be shown as implemented database fields if the corresponding table exists in the current schema.

Isolation Forest and Random Forest have been considered for anomaly analysis, but the operational performance of the anomaly-detection component has not been fully validated. Therefore, anomaly history should be documented as a planned or conditional entity unless the actual database implementation confirms its existence.

5.5.4 Relationship Between Entities

The logical database model relates monitoring records to replay-management information and, where implemented, anomaly history.

A telemetry record is associated with the message nonce used during authenticated communication. The nonce is interpreted as an eight-byte session identifier followed by a four-byte counter in the current replay-protection implementation. The backend uses the session and counter values to determine whether a message should be accepted.

The replay-state tables maintain information about accepted sessions and counters. The precise foreign-key relationships between these tables and the telemetry table must be derived from the actual database schema rather than assumed.

If anomaly history is implemented as a persistent entity, an anomaly result may refer to the telemetry record from which it was derived. The relationship should be represented using a foreign key only if that constraint exists in the database.

ER Diagram — diagram specification

The ER diagram should show the implemented telemetry table and the replay-protection tables `secure_bms_replay_sessions` and `secure_bms_replay_state`. Mark primary keys, foreign keys, and cardinalities only after checking the database schema. If an Anomaly History entity is included before its implementation is confirmed, label it as a proposed entity rather than an existing database table.

5.6 UI Design / Wireframes

The user interface is intended to provide a centralized view of the battery-monitoring information collected by Secure-BMS. It should allow the operator to inspect the latest available telemetry, identify configured alert conditions, and review relevant diagnostic information.

The current dashboard is implemented using Streamlit. The UI design should therefore represent the available dashboard functions and distinguish them from proposed interface features that have not yet been implemented.

The dashboard is organized conceptually into the following areas:

1. System Status Panel

This panel presents the reported operating state, such as `NORMAL`, `TEMPERATURE_ALERT`, or `SENSOR_FAULT`, where the corresponding information is available.

2. Temperature Monitoring Panel

This panel displays the estimated temperature derived from the thermistor measurement. A historical chart may be included if the dashboard is configured to retrieve and display stored temperature records.

3. Telemetry and Diagnostic Panel

This panel presents available measurement fields and diagnostic statistics, including supported alert and sensor-fault information. Unvalidated measurements should not be presented as verified values.

4. Communication and Data Status Panel

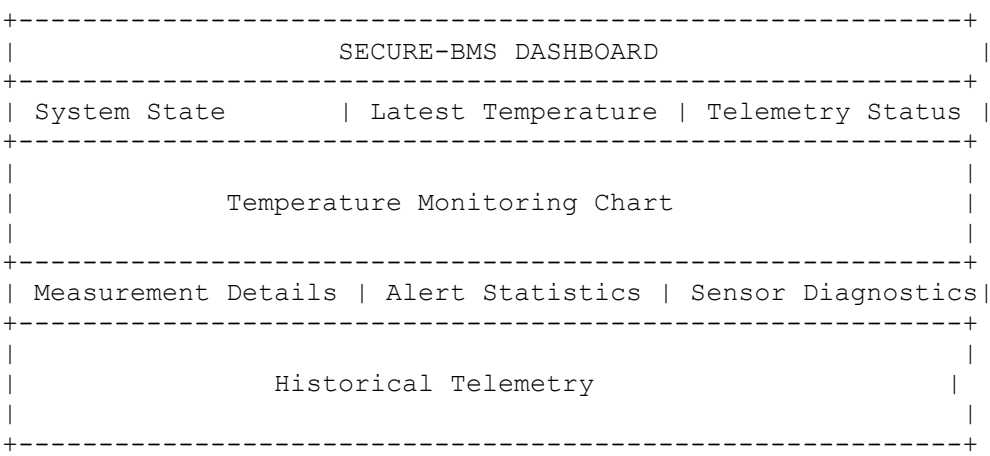
This panel indicates the availability of relevant telemetry or backend information where such status data is exposed by the implementation. It should not imply that every communication component is operational when its status has not been checked.

5. Historical Monitoring Panel

This area is intended to support inspection of previously stored telemetry. Its displayed values depend on the database queries and visualizations implemented in the dashboard.

Proposed Wireframe Layout

The conceptual layout is shown below.



The wireframe represents a proposed logical arrangement rather than a screenshot of the current application. The final report should include an actual screenshot of the implemented Streamlit dashboard if one is available.

Remote relay control, authenticated operator commands, and real-time machine-learning classifications should not be presented as active dashboard capabilities unless they have been implemented and verified.

Chapter Summary

This chapter described the Secure-BMS system architecture, use case model, logical class structure, interaction workflows, database design, and dashboard layout. The design separates physical measurement acquisition, embedded processing, encrypted telemetry communication, backend validation, database persistence, and user-interface presentation.

The diagrams should be prepared using the actual hardware connections, source code, and database schema. Implemented functions must be distinguished from proposed extensions, particularly automated load switching, persistent anomaly history, and validated machine-learning analysis. This ensures that the system-design documentation remains consistent with the current prototype.

6. Implementation

6.1 Introduction

The implementation phase of Secure-BMS involves translating the system architecture and requirements into an operational hardware and software prototype. The project integrates an ESP32-based embedded monitoring system with secure MQTT communication, backend message processing, PostgreSQL storage, and a Streamlit dashboard.

The implementation follows a modular approach in which the hardware-monitoring functions are separated from the communication and data-processing functions. The embedded controller acquires sensor measurements, evaluates supported alert conditions, and prepares

telemetry. The backend receives encrypted messages, verifies their authenticity, checks for replay attempts, and stores accepted records.

The implemented prototype provides a foundation for secure battery monitoring and further development of cyber-physical security mechanisms. However, the implementation status of each subsystem differs. Temperature monitoring, local alert logic, encrypted telemetry processing, and database-level replay protection have been developed, while complete end-to-end integration testing, validated machine-learning inference, and automated load switching remain outside the verified capabilities of the current configuration.

6.2 Hardware Implementation

6.2.1 ESP32 Microcontroller Setup

The ESP32 NodeMCU development board serves as the primary embedded controller. It is responsible for sensor communication, measurement processing, event-state evaluation, local alert handling, and wireless telemetry transmission.

The firmware is developed using the Arduino IDE and the ESP32 Arduino platform. The controller communicates with the ADS1115 analog-to-digital converter through the I²C interface, using GPIO21 as SDA and GPIO22 as SCL in the configured firmware.

The board also interfaces with the comparator and buzzer circuit through dedicated GPIO pins. Wi-Fi connectivity enables the transmission of monitoring information to the software backend through MQTT.

6.2.2 Temperature Sensing Circuit

Temperature monitoring is implemented using an NTC thermistor with an approximate nominal resistance of 100 kΩ and a B3950 thermistor constant. The thermistor is connected as part of a voltage-divider circuit with a 10 kΩ resistor.

The voltage-divider output is connected to the ADS1115, which communicates with the ESP32 over I²C. The ADS1115 was detected at address `0x48` during hardware testing.

The ESP32 processes the digital readings and estimates the corresponding temperature using the thermistor parameters. A nine-sample median filter is implemented to reduce the influence of isolated measurement fluctuations. This improves resistance to individual noisy readings, although measurement accuracy still depends on the thermistor characteristics, circuit configuration, and calibration.

6.2.3 Comparator and Local Alert Circuit

The LM339 comparator is used to provide a hardware threshold-condition signal. A potentiometer allows adjustment of the switching threshold, and the comparator output is connected to ESP32 GPIO26.

An active buzzer is connected to GPIO25 and is controlled using an active-low signal. The comparator and firmware provide the inputs required for the configured alert logic.

The local alert circuit allows the prototype to indicate potentially abnormal conditions without depending exclusively on remote telemetry. The threshold must be calibrated for the intended experimental conditions, and the alert circuit must not be interpreted as a substitute for a certified battery protection mechanism.

6.2.4 Load-Switching Hardware

An IRLZ44N MOSFET is included in the hardware configuration, with its gate connected to ESP32 GPIO27. The component forms part of the intended load-switching arrangement.

However, the current firmware configuration uses `LOAD_ENABLE = false`. Therefore, automated load switching is not treated as an active or verified protection function in this report. The presence of the MOSFET in the circuit does not establish that automatic battery disconnection is operational.

6.2.5 Battery and Supporting Components

The prototype uses an 18650 lithium-ion cell labelled 3.7 V, 2000 mAh, and 7.4 Wh. Supporting hardware includes the battery charging and protection modules, breadboard, connecting wires, USB cable, and multimeter.

Basic electrical checks and sensor tests were performed during development. The INA219 module was detected at I²C address `0x40`, but its current and voltage measurement path was not fully validated. Consequently, its readings are not treated as verified current-monitoring results.

The hardware implementation remains an experimental single-cell testbed. Electrical connections, charging arrangements, and load operation must follow the relevant component specifications and suitable battery-safety procedures.

6.3 Embedded Firmware Implementation

6.3.1 Firmware Development Environment

The embedded firmware is developed in C++ using the Arduino IDE and the ESP32 Arduino platform. The firmware integrates the sensor interface, measurement processing, event-state logic, local alert handling, wireless connectivity, and encrypted telemetry generation.

The firmware uses the ADS1115 interface library and the relevant Wi-Fi, MQTT, and cryptographic functionality. Sensitive cryptographic configuration is maintained separately from publicly shared source code.

6.3.2 Sensor Acquisition and Filtering

The ESP32 obtains sensor measurements from the ADS1115 through the I²C interface. The thermistor circuit produces a voltage that varies with temperature, and the firmware processes the acquired readings to estimate temperature.

The implementation uses a median filter over nine samples, with a 10 ms delay between successive samples in the filtering routine. The median value is used to reduce the influence of isolated readings before subsequent temperature calculations.

The filtered measurement is converted into an estimated temperature using the configured thermistor model. The quality of this estimate depends on the correctness of the resistance-divider arrangement, thermistor parameters, and calibration.

6.3.3 Event-State Management

The firmware evaluates supported measurement and hardware conditions and represents the operating condition using event states. The implemented state names include:

- `NORMAL`
- `TEMPERATURE_ALERT`
- `SENSOR_FAULT`

The `NORMAL` state represents operation without a currently detected configured alert or supported sensor fault. `TEMPERATURE_ALERT` represents a detected temperature-related alert condition, while `SENSOR_FAULT` represents a condition identified by the implemented fault-checking logic.

The firmware also maintains monitoring statistics, including peak temperature, minimum and maximum voltage fields, alert count, alert-duration information, and sensor-fault count. The accuracy of individual fields depends on the validity of their underlying measurement sources.

6.3.4 Wi-Fi Connectivity and Recovery

Wi-Fi connectivity is implemented in the ESP32 firmware to support remote telemetry transmission. Reconnection logic attempts to restore network connectivity when the connection is interrupted.

Network availability is required for publishing telemetry to the MQTT broker. A loss of Wi-Fi can interrupt remote monitoring, although it does not necessarily imply that every local hardware-monitoring function has stopped.

The firmware's reconnection mechanism supports recovery from supported connectivity interruptions, but uninterrupted network operation is not guaranteed.

6.4 Secure Telemetry Implementation

6.4.1 AES-128-GCM Encryption

Secure-BMS implements AES-128-GCM to protect telemetry before transmission. AES-GCM provides confidentiality and authentication through encryption and an authentication tag.

The firmware uses the Mbed TLS cryptographic implementation. Each telemetry message is encrypted with a nonce, and the MQTT topic is supplied as additional authenticated data (AAD). This binds the authentication operation to the expected topic.

The message envelope contains the following fields:

Field	Description
`enc`	Identifies the encryption algorithm as `A128GCM`
`nonce`	Contains the Base64-encoded cryptographic nonce
`ciphertext`	Contains the Base64-encoded encrypted payload and authentication tag

The receiving backend uses the corresponding cryptographic configuration to authenticate and decrypt the message. A message that fails authentication must not be processed as valid telemetry.

6.4.2 Nonce Generation and Message Counter

The firmware generates a 12-byte nonce consisting of an eight-byte random prefix associated with the boot session and a four-byte message counter encoded in big-endian order.

The counter is incremented before encryption, and the firmware refuses further counter use when `messageCounter == UINT32_MAX`, preventing counter wraparound in the implemented logic.

The session identifier and counter support the backend's replay-protection mechanism. Secure nonce generation and correct key management are essential because reuse of a nonce with the same AES-GCM key can compromise cryptographic security.

6.4.3 MQTT Telemetry Publication

The ESP32 publishes encrypted telemetry using MQTT. The configured telemetry topic is `secure-bms/telemetry`, and the local Mosquitto broker is used as the message-broker component.

MQTT separates the embedded publisher from the backend subscriber. The ESP32 prepares and encrypts telemetry, while the backend receives messages independently through its MQTT client.

The current implementation uses a local broker environment. The presence of AES-GCM encryption does not by itself establish that all network transport, broker authentication, access-control, and deployment-security requirements have been satisfied.

6.5 Backend Implementation

6.5.1 Backend Architecture

The backend is implemented in Python and uses FastAPI, a Paho MQTT client, and PostgreSQL connectivity through Psycopg.

The backend is responsible for receiving MQTT messages, authenticating and decrypting telemetry, validating the payload, applying replay-protection checks, and storing accepted records.

The implementation separates message reception from cryptographic validation and database persistence. This structure helps ensure that a received message is not automatically treated as trusted data.

6.5.2 Message Decryption and Validation

The function `decode_telemetry_payload(payload, topic)` processes the encrypted message envelope. It validates the required envelope fields, decodes the nonce and ciphertext, and uses AES-128-GCM to authenticate and decrypt the message.

The MQTT topic is supplied as authenticated additional data. If the topic, ciphertext, nonce, or authentication tag does not satisfy the expected conditions, the message must not be accepted as valid telemetry.

After successful decryption, the backend validates the payload structure and extracts the information required for persistence. The message handler includes error handling for invalid authentication tags, replay-detection failures, malformed values, missing keys, and JSON decoding errors.

6.5.3 MQTT Message Handling

The backend MQTT callback processes received messages through a validation pipeline. The message is first passed to the decryption and authentication function. The resulting data is then subjected to payload validation and database-level replay checks before it is accepted.

The message counter is incremented only after the telemetry has been saved successfully. This prevents rejected or unsuccessfully stored messages from being counted as successfully persisted telemetry.

The callback handles supported validation and processing errors without treating failed messages as accepted records.

6.6 Replay-Protection Implementation

6.6.1 Replay-Protection Mechanism

Replay protection is implemented in the backend using the nonce generated by the ESP32. The 12-byte nonce is interpreted as an eight-byte session identifier followed by a four-byte message counter.

The backend checks the session and counter information against persistent database state. For an already active session, a counter that is not greater than the last accepted counter is rejected. The implementation also rejects a session that has previously been seen but is no longer considered an acceptable new session under the replay rules.

This mechanism is intended to prevent duplicate or stale telemetry from being accepted as new data.

6.6.2 Database-Backed Replay State

PostgreSQL stores replay-management information in the tables `secure_bms_replay_sessions` and `secure_bms_replay_state`.

The backend uses transaction-based processing to coordinate replay validation and telemetry insertion. The replay-state row is checked and updated as part of the same transaction used to store accepted telemetry.

If the message is rejected or a database operation fails, the transaction is rolled back as appropriate. This prevents the replay state from being advanced independently of a successful telemetry insertion.

6.6.3 Replay-Protection Testing

Unit-level and database-level tests have been performed for the following scenarios:

1. Acceptance of a fresh message.
2. Acceptance of a message with an increasing counter.
3. Rejection of duplicate or stale counters.
4. Acceptance of a valid new session.
5. Rejection of a previously seen inactive session.
6. Transaction rollback when the database operation fails.

These tests provide evidence for the replay-validation logic and database consistency. End-to-end replay testing through the live MQTT callback remains incomplete; therefore, full system-level replay resistance should not be claimed as conclusively verified.

6.7 Database Implementation

6.7.1 PostgreSQL Integration

PostgreSQL is used to persist accepted telemetry and replay-protection state. The Python backend connects to the database using Psycopg.

The database layer supports the storage of validated monitoring data and the session and counter information required by the replay-protection mechanism. Persistent storage enables previously received message information to remain available across backend operations.

The backend performs the relevant database checks before considering a telemetry message successfully stored.

6.7.2 Transaction Management

Database transactions are used to preserve consistency between replay validation, replay-state updates, and telemetry insertion.

The backend checks the nonce against the stored state and proceeds with persistence only when the message satisfies the replay rules. The telemetry record and corresponding replay-state changes are handled within a transaction so that a failure can trigger rollback.

This approach reduces the risk of inconsistent state in which a message counter is recorded as accepted without its corresponding telemetry record being stored.

6.7.3 Data Integrity

The database implementation is designed to prevent invalid or rejected messages from being represented as accepted telemetry. The backend validates messages before persistence, and transaction management helps preserve consistency when operations fail.

The exact telemetry columns, database constraints, and entity relationships must be documented according to the actual database schema. Proposed anomaly-history records should not be described as implemented database tables unless their existence is confirmed.

6.8 Dashboard Implementation

6.8.1 Streamlit Dashboard

The monitoring interface is implemented using Streamlit. It is intended to provide a convenient way to inspect stored telemetry and system-status information without requiring the operator to interact directly with low-level firmware output or database queries.

The dashboard forms the presentation layer of the software architecture. Its information depends on the telemetry available through the backend and database.

6.8.2 Monitoring Information

The dashboard is intended to present available temperature measurements, reported event states, and relevant monitoring statistics. Historical monitoring views depend on the database queries and visualizations implemented in the application.

The interface should distinguish valid measurements from unavailable or unvalidated values. In particular, current or voltage data from the INA219 should not be presented as verified measurements until the corresponding hardware path has been validated.

A dashboard screenshot should be included in the final report as implementation evidence if the running interface is available.

6.8.3 Integration Limitations

The complete monitoring path depends on the availability of the MQTT broker, backend service, database, and dashboard. The dashboard's intended role does not establish that every proposed visualization or advanced analytical function has been completed.

Machine-learning classifications, remote hardware control, and other advanced dashboard functions must be treated as incomplete unless their implementation and behaviour have been verified.

6.9 Anomaly-Detection Component

Machine-learning-based anomaly detection is intended to extend Secure-BMS by identifying unusual patterns in battery telemetry. Isolation Forest and Random Forest have been considered for the analytical component.

Isolation Forest can be investigated for identifying observations that differ from typical measurement patterns, while Random Forest can be used for classification when suitable labelled training data and a defined target are available.

The anomaly-detection worker is included in the software architecture, but the operational model, training data, evaluation metrics, and detection performance have not been fully validated. Consequently, the current implementation cannot be claimed to provide verified machine-learning-based fault prediction.

Further work should establish the input features, training and testing datasets, model-selection procedure, evaluation metrics, and integration with the telemetry pipeline. The output should also be distinguished from hardware-generated alerts and should not be used as the sole basis for battery-safety decisions.

6.10 Software Integration

The software integration process connects the embedded controller to the backend through the MQTT communication layer. It also connects the backend to PostgreSQL and the Streamlit dashboard.

The integration workflow comprises the following stages:

1. The ESP32 acquires and processes sensor measurements.
2. The firmware prepares and encrypts the telemetry message.
3. The MQTT publisher sends the encrypted message to the configured topic.
4. The Mosquitto broker forwards the message to the backend subscriber.
5. The backend authenticates, decrypts, and validates the payload.
6. The replay-protection logic checks the session and message counter.
7. Accepted telemetry and replay-state changes are stored transactionally in PostgreSQL.
8. The dashboard presents the available monitoring information.

Each stage depends on the correct configuration of the preceding components. A failure in connectivity, authentication, validation, or persistence can interrupt the remote monitoring pipeline. End-to-end integration testing is therefore necessary to establish that the complete workflow behaves correctly under normal and abnormal conditions.

6.11 Implementation Status

Table 6.1 summarizes the implementation status of the principal Secure-BMS components.

Table 6.1: Implementation Status of Secure-BMS

Component	Implementation status
ESP32 firmware and Wi-Fi reconnection	Implemented
ADS1115 temperature-measurement interface	Implemented and tested
Nine-sample median filtering	Implemented
Comparator-based alert input	Implemented
Active-low buzzer control	Implemented
MOSFET hardware connection	Present; automated load switching disabled
AES-128-GCM telemetry encryption	Implemented
MQTT telemetry publishing	Implemented in the firmware
Backend authenticated decryption	Implemented
Backend payload validation	Implemented
PostgreSQL telemetry persistence	Implemented in the backend
Database-backed replay protection	Implemented and tested at unit/database level
End-to-end replay testing through live MQTT callback	Pending
INA219 current and voltage measurement validation	Incomplete
Streamlit monitoring dashboard	Included in the software implementation; full integration must be verified
Operational machine-learning anomaly detection	Requires further validation

The table differentiates implemented functions from those that require additional verification. It should be updated if subsequent testing establishes new results.

6.12 Chapter Summary

This chapter described the implementation of the Secure-BMS hardware and software components. The hardware implementation integrates the ESP32 controller, ADS1115-based thermistor measurement, comparator-based threshold input, and local audible alert. The embedded firmware processes sensor measurements, maintains event states, prepares telemetry, and supports Wi-Fi reconnection.

The software implementation uses AES-128-GCM for telemetry encryption and authentication, MQTT for message transport, Python-based backend processing, PostgreSQL for telemetry and replay-state persistence, and Streamlit for monitoring. The replay-protection mechanism checks session and counter information and uses database transactions to maintain consistency between replay-state updates and telemetry insertion.

The implementation establishes the principal components of the Secure-BMS prototype. Further work is required to complete end-to-end replay testing, validate the INA219 measurement path, verify complete dashboard integration, evaluate anomaly-detection

performance, and test additional hardware and system-level failure conditions. Automated load switching remains disabled in the current firmware configuration.

7. Software Tools Used (Demo Screenshots)

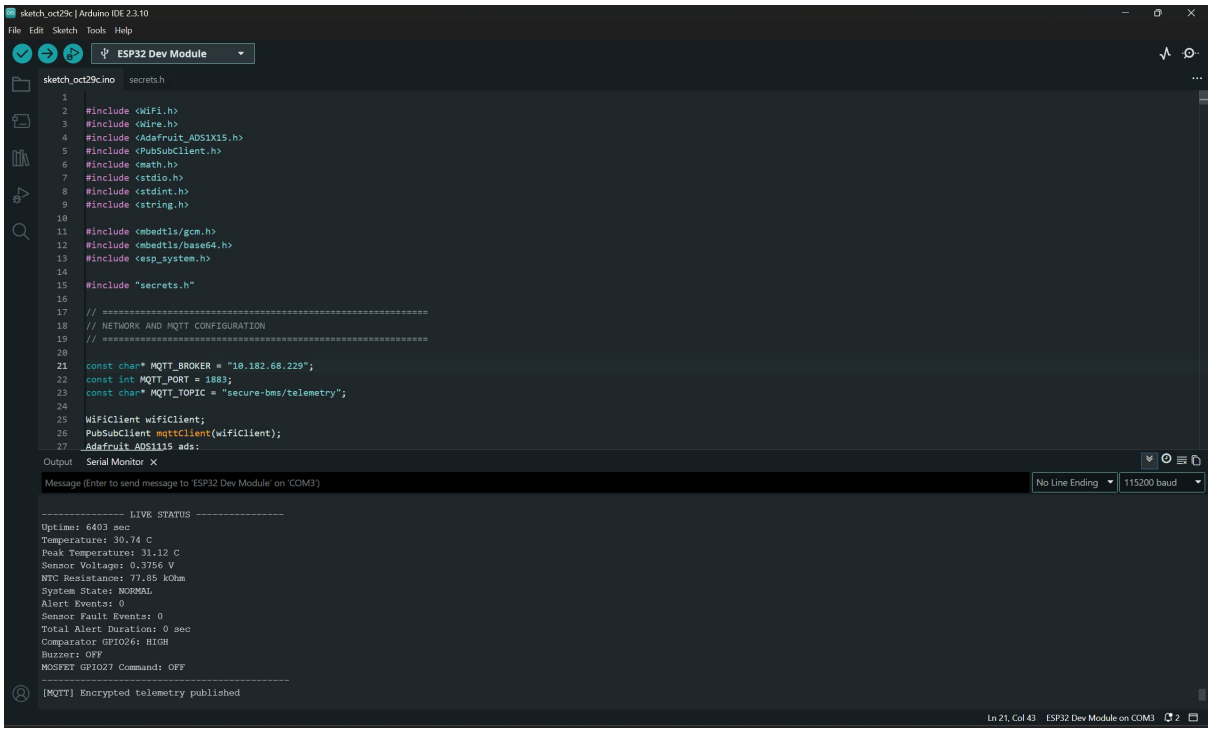
7.1 Arduino IDE

Arduino IDE was used as the development environment for programming and uploading firmware to the ESP32 microcontroller. It supports writing, compiling, and uploading embedded C/C++ code to the development board.

In Secure-BMS, the firmware reads temperature-sensor data through the ADS1115 analog-to-digital converter, processes the readings, manages temperature and sensor-fault states, controls the local buzzer output, and establishes Wi-Fi connectivity for telemetry communication.

The Serial Monitor was used to observe diagnostic messages and sensor readings during development and testing.

Demo screenshot: Arduino IDE displaying the Secure-BMS firmware, with the compilation or upload result and Serial Monitor output where available.

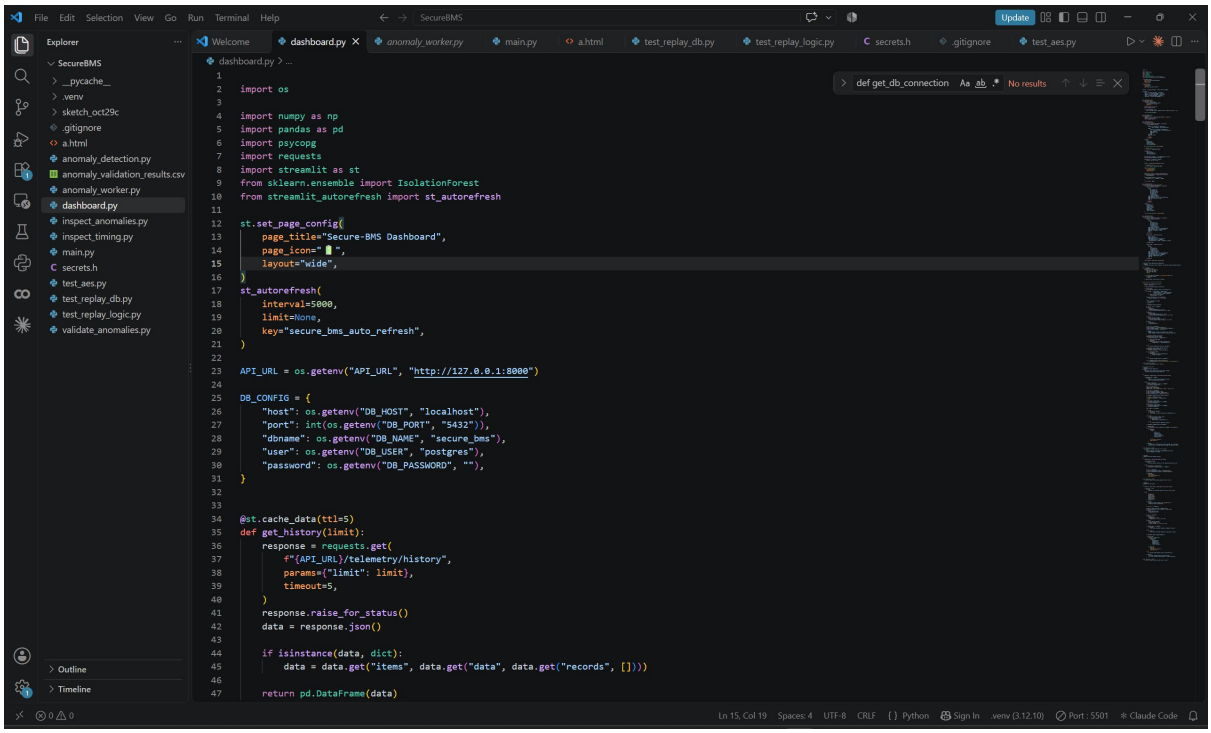


7.2 Visual Studio Code

Visual Studio Code was used as the development environment for the software components running on the computer. It provides source-code editing, project navigation, terminal access, and debugging support.

For Secure-BMS, it is suitable for organizing and maintaining the backend and dashboard code, including the FastAPI application, MQTT message-processing logic, database operations, and Streamlit dashboard. It also helps manage the Python files associated with telemetry processing and anomaly detection.

Demo screenshot: Visual Studio Code showing the Secure-BMS software project and a relevant backend or dashboard source file.



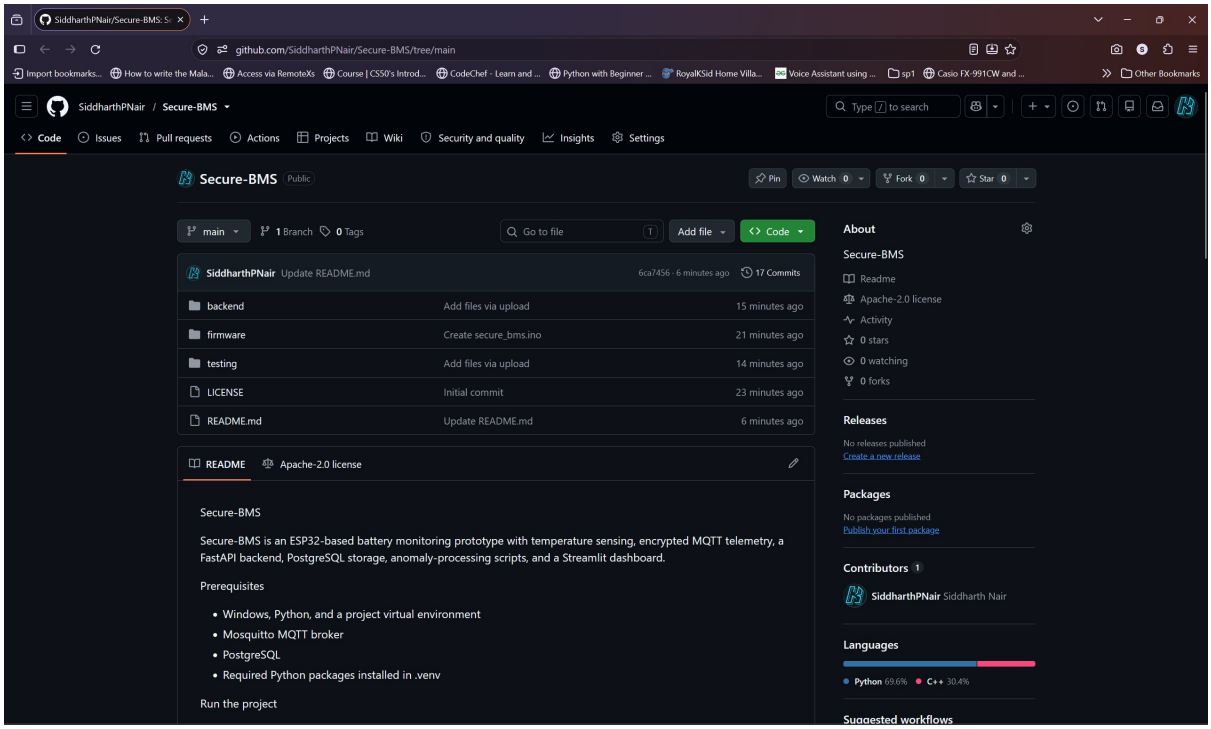
7.3 Git and GitHub

Git is a version-control system used to track source-code changes and maintain the development history of a software project. GitHub provides a remote platform for storing and managing Git repositories.

For Secure-BMS, these tools support organized source-code management, change tracking, and backup of project files. Version control is particularly useful when modifying embedded firmware, telemetry-security logic, replay-protection mechanisms, and dashboard functionality.

Sensitive information, including cryptographic keys, passwords, and other credentials, should not be committed to a public repository. Such values should be managed separately from the source code.

Demo screenshot: The Secure-BMS repository or a terminal displaying relevant Git commands and repository status.



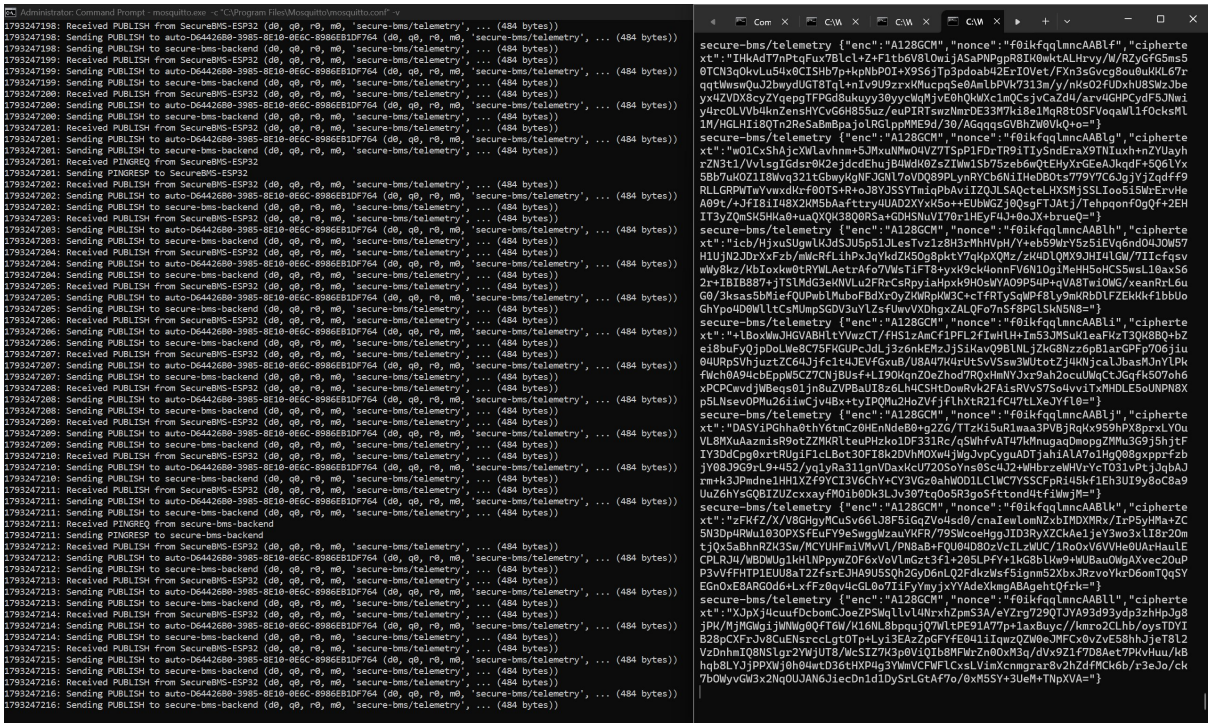
7.4 Mosquitto MQTT Broker

Eclipse Mosquitto is a lightweight MQTT broker that facilitates communication between MQTT publishers and subscribers. It routes messages published to a topic to clients subscribed to that topic.

In Secure-BMS, the ESP32 publishes encrypted telemetry to the `secure-bms/telemetry` topic. The backend subscribes to this topic, receives the encrypted message envelope, authenticates and decrypts the payload, and processes valid telemetry for storage.

Mosquitto provides the local message-broker component of the system. The broker transports messages, while payload encryption and replay validation are handled by the Secure-BMS application logic.

Demo screenshot: The Mosquitto broker running on the development computer, accompanied by relevant MQTT publisher or subscriber output if available.



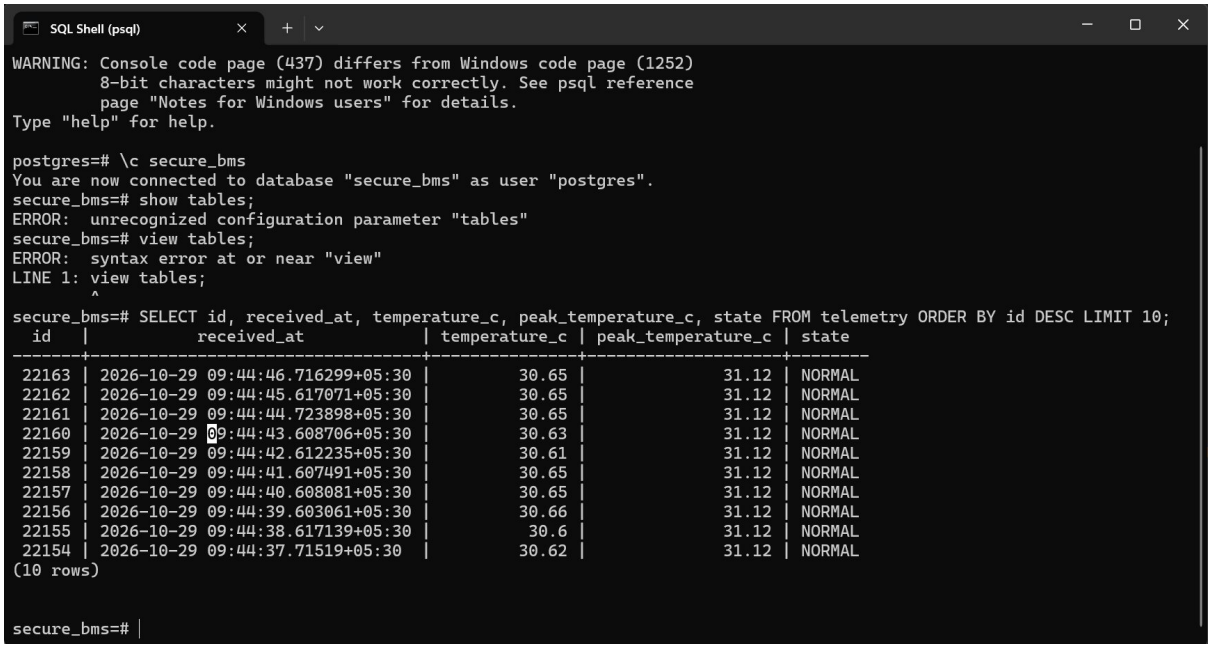
7.5 PostgreSQL

PostgreSQL is a relational database management system used to store structured application data. It supports SQL queries, transactions, constraints, and consistent database updates.

In Secure-BMS, PostgreSQL stores processed telemetry and maintains database-backed state used for replay protection. The replay-protection mechanism tracks message sessions and counters to help reject duplicate or stale telemetry messages.

Database transactions are important because telemetry insertion and replay-state updates must remain consistent. If an operation fails, the transaction should be rolled back to prevent partially applied updates.

Demo screenshot: PostgreSQL database inspection showing the Secure-BMS database and relevant telemetry or replay-protection tables. Use the actual table names and records from the implemented database.



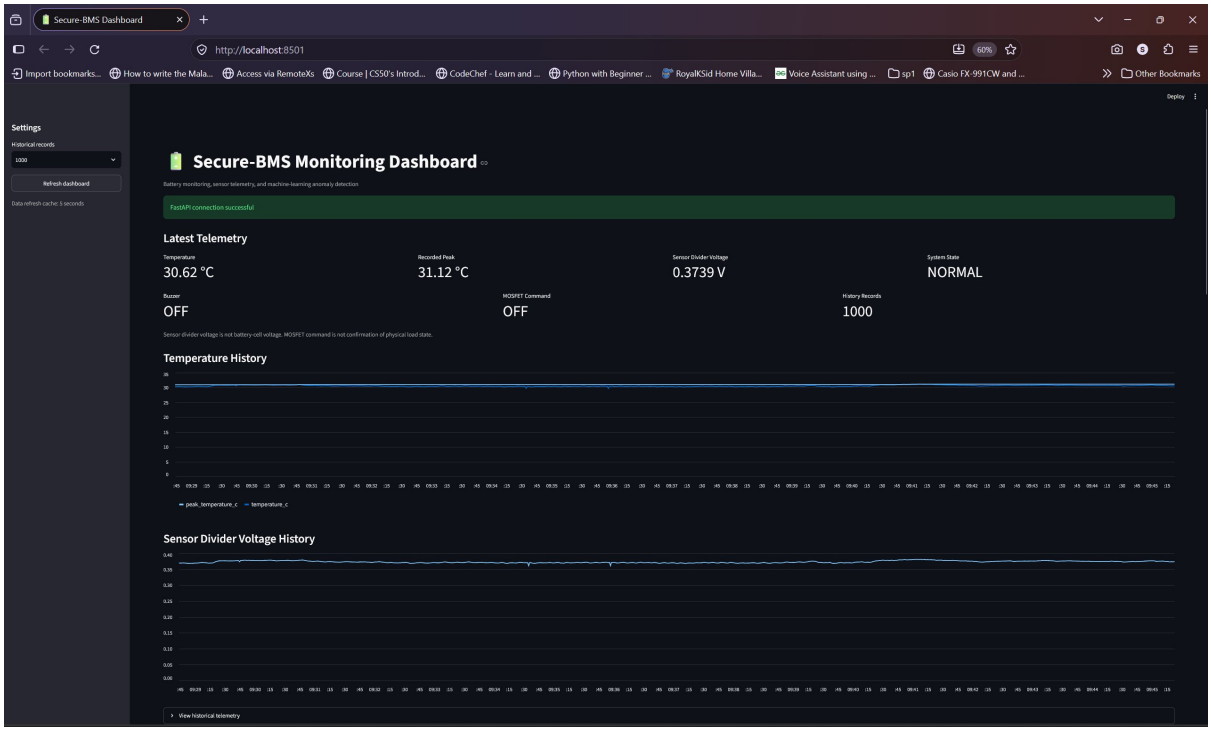
7.6 Streamlit

Streamlit is a Python framework for developing interactive data applications. It allows developers to build dashboards using Python without requiring a separate frontend framework.

In Secure-BMS, Streamlit is used for the monitoring dashboard. It provides a user interface for presenting available telemetry and system-status information retrieved from the backend or database.

The dashboard supports the project's monitoring objective by making processed system information easier to inspect. Its displayed values and features depend on the functionality implemented and connected to the current data pipeline.

Demo screenshot: The running Secure-BMS Streamlit dashboard showing the telemetry fields and status indicators currently supported by the application.



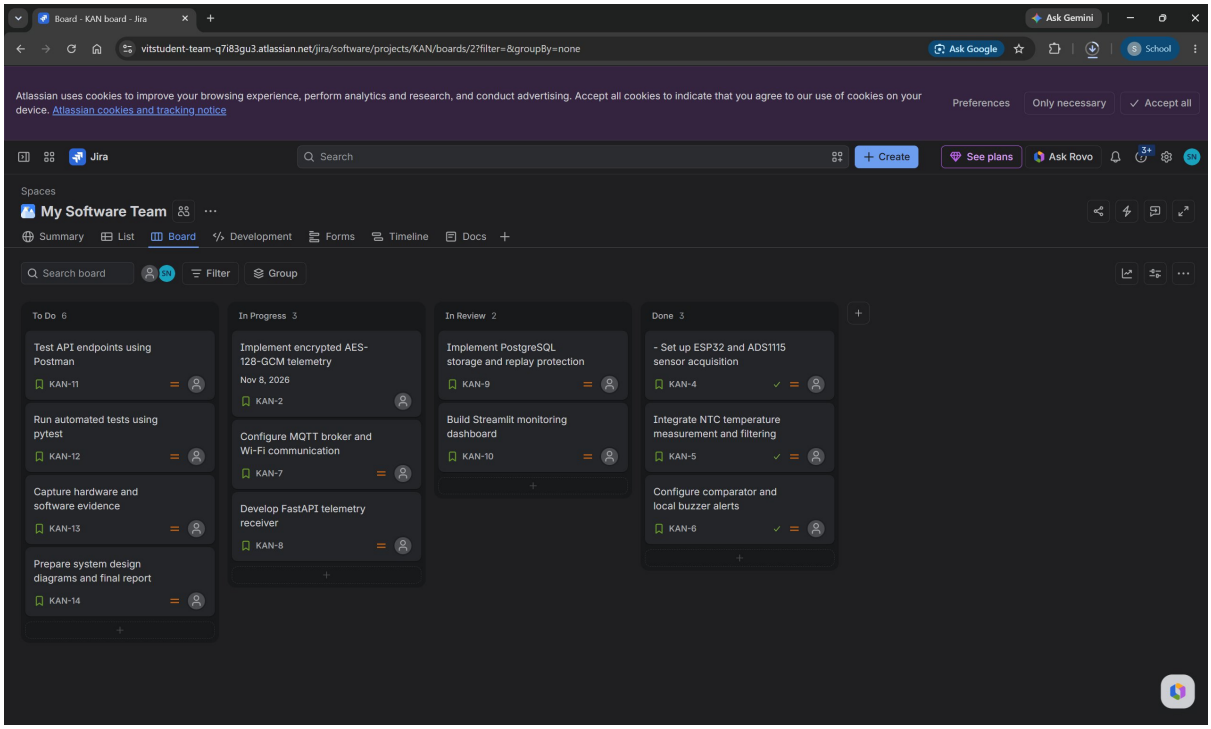
7.7 Jira

Jira is a project-management tool used to organize tasks, track progress, and manage development activities.

For Secure-BMS, Jira can be used to divide the work into manageable tasks, such as hardware integration, firmware development, encrypted MQTT communication, backend processing, database integration, and dashboard development. Task tracking helps monitor progress and identify incomplete activities.

The project board should reflect the actual tasks and workflow used by the team.

Demo screenshot: The Secure-BMS Jira project board or task list, if maintained during development.



Chapter Summary

This chapter describes the software tools associated with Secure-BMS development, testing, communication, data storage, monitoring, and project management. Arduino IDE supports ESP32 firmware development, Visual Studio Code supports software development, Git and GitHub support version control, Mosquitto enables MQTT communication, PostgreSQL stores structured data, Streamlit provides the dashboard interface. Jira supports task organization where used.

The screenshots included in the final report should demonstrate the actual tools and project artifacts used during implementation. They should be captured from the working environment and should not expose passwords, cryptographic keys, or other sensitive configuration values.

8. Testing

8.1 Testing Strategy

Testing was carried out to evaluate the functionality, reliability, and integration of the Secure-BMS (Secure Battery Management System) prototype. The testing strategy focused on verifying the hardware components, embedded firmware, encrypted telemetry communication, backend processing, database operations, and monitoring dashboard.

The system was tested in stages to make it easier to identify faults and isolate problems. Individual components were checked before evaluating their interaction with other parts of the system.

The following testing levels were considered:

1. **Component Testing:** Individual hardware and software components were checked independently, including the ESP32, temperature sensor, analog-to-digital converter, comparator, buzzer, and firmware functions.
2. **Functional Testing:** The system's response to temperature readings, temperature alerts, sensor faults, and telemetry processing was examined.
3. **Communication Testing:** Wi-Fi connectivity and MQTT message transmission were evaluated as part of the telemetry pipeline.
4. **Security Testing:** AES-128-GCM authenticated encryption and database-backed replay-protection logic were examined through software-level tests.
5. **Database Testing:** Telemetry storage and replay-state updates were checked for data consistency and transactional behavior.
6. **Integration Testing:** The interaction between the embedded device, MQTT broker, backend, database, and dashboard was considered.

Testing was performed with the objective of confirming the prototype's implemented functionality. A successful individual component test does not necessarily establish that the complete end-to-end system works under every operating condition.

8.2 Test Cases

Test cases were prepared to verify the expected behavior of the hardware and software components. Each test case specifies the functionality being examined, the test procedure, and the result or evidence required.

The following table summarizes the principal test cases relevant to Secure-BMS.

Table 8.1: Secure-BMS Test Cases

Test ID	Test case	Test procedure	Expected result
TC-01	ESP32 initialization	Power the ESP32 and inspect serial output.	The firmware initializes successfully and begins execution.
TC-02	ADS1115 detection	Check the I²C address of the ADC.	The ADS1115 is detected at address `0x48`.
TC-03	Temperature measurement	Observe the thermistor readings under different temperature conditions.	The measured temperature changes in response to the sensor conditions.
TC-04	Temperature alert	Increase the sensor temperature and observe the comparator and firmware response.	The alert condition is detected when the configured threshold is crossed.
TC-05	Buzzer operation	Observe the buzzer output during alert conditions.	The buzzer operates according to the configured active-low control logic.
TC-06	Sensor-fault handling	Examine firmware behavior when a sensor reading is invalid or a fault is detected.	The firmware identifies the fault and reports the appropriate state.
TC-07	Wi-Fi connectivity	Start the device and observe its network connection	The device attempts to connect and reconnect when connectivity is

		behavior.	interrupted.
TC-08	MQTT telemetry	Publish encrypted telemetry and observe the broker and subscriber behavior.	Messages are transported through the configured MQTT topic.
TC-09	Authenticated decryption	Submit a valid encrypted telemetry envelope to the backend.	A valid message can be authenticated and decrypted.
TC-10	Invalid encrypted message	Submit an invalid or incorrectly authenticated payload to the decoder.	The payload is rejected and is not processed as valid telemetry.
TC-11	Replay protection	Test fresh, increasing, duplicate, and stale message counters.	Valid new messages are accepted, while duplicate or stale counters are rejected.
TC-12	Session validation	Test a new session and a previously seen inactive session.	A valid new session is handled according to policy, and a previously seen inactive session is rejected.
TC-13	Database transaction	Exercise telemetry insertion and replay-state updates, including failure handling.	The transaction preserves consistency and rolls back failed operations.
TC-14	Dashboard display	Open the Streamlit dashboard and inspect available telemetry.	The dashboard presents the information supported by its current implementation.
TC-15	Anomaly detection	Execute the implemented anomaly-detection component using suitable input data.	The component produces results according to its implemented model and configuration.

The expected results above define the verification criteria; they should not be interpreted as proof that every test passed. Actual outcomes must be recorded from the corresponding test execution.

Software-level replay-protection tests were conducted for fresh messages, increasing counters, duplicate or stale counters, new sessions, previously seen sessions, and transaction rollback. These tests provided evidence for the replay-protection and database logic. However, end-to-end replay testing through the live MQTT callback was not completed and should be treated as pending integration validation.

8.3 Hardware Testing Results

Hardware testing focused on the operation of the ESP32 development board, temperature-sensing circuit, comparator, buzzer, and load-control interface.

8.3.1 ESP32 and ADC Testing

The ESP32 NodeMCU development board was programmed through the Arduino IDE. Serial communication was used to observe firmware diagnostics and sensor readings.

The ADS1115 analog-to-digital converter was detected at I²C address `0x48`. The firmware uses the ADC to acquire the thermistor-divider voltage. Multiple samples are collected and median filtering is applied to reduce the influence of isolated reading variations.

The sensor was tested under different conditions, including warming the thermistor by touch. The reported temperature responded to the change in sensor conditions, demonstrating that the acquisition circuit was producing usable readings during the test.

8.3.2 Temperature-Sensing Testing

The temperature-sensing circuit uses a nominally 100 kΩ NTC thermistor with a B3950 characteristic and a 10 kΩ fixed resistor in a voltage-divider arrangement.

The firmware calculates an estimated temperature from the measured voltage and the configured thermistor parameters. The readings were observed to change when the thermistor temperature changed.

These observations support the basic operation of the sensing circuit. However, they do not constitute a calibrated accuracy assessment because the measurements were not independently validated across a reference temperature range.

8.3.3 Comparator and Buzzer Testing

The LM339 comparator output is connected to ESP32 GPIO26. The comparator was observed to change its output state when the sensor condition crossed the configured threshold. The active-low buzzer is controlled through GPIO25.

During testing, the comparator output changed between the alert and non-alert states as the temperature condition changed. The buzzer was also observed to operate according to the configured control logic.

These tests demonstrated the basic operation of the local alert path. Further testing with controlled temperature references would be required to quantify threshold accuracy, response time, and repeatability.

8.3.4 Load-Control Interface

The hardware includes an IRLZ44N MOSFET with its gate connected to ESP32 GPIO27. The firmware baseline retains `LOAD_ENABLE = false`, meaning automated load switching is not enabled in the current firmware configuration.

Consequently, the presence of the MOSFET interface should not be interpreted as proof of an operational automatic load-disconnection function. Such functionality would require separate validation, including switching behavior, failure handling, and safe operation under representative electrical conditions.

8.3.5 Hardware Testing Summary

The component checks established that the ESP32, ADS1115, temperature-sensing circuit, comparator, and buzzer could be operated during development. The tests were prototype-level functional checks rather than formal electrical-safety or battery-certification tests.

The prototype should therefore be considered an experimental monitoring system. It has not been established as a certified battery protection system.

8.4 Defects and Limitations Identified

Testing and integration identified limitations that affect the scope of the conclusions that can be drawn from the prototype.

8.4.1 Temperature Measurement Accuracy

The temperature is calculated from the thermistor-divider voltage and configured thermistor parameters. Although the readings responded to changes in sensor conditions, comprehensive calibration against a reference thermometer was not completed.

Limitation: Absolute measurement accuracy across the operating temperature range has not been established.

8.4.2 Current Measurement Validation

The INA219 sensor was detected at I²C address `0x40`, but its current and voltage measurements were not fully validated. Current measurement should therefore not be presented as a verified capability of the completed prototype.

Limitation: Accurate current and power monitoring require additional validation of the sensor wiring, measurement configuration, and readings against a suitable reference.

8.4.3 End-to-End Security Validation

AES-128-GCM authenticated encryption and database-backed replay-protection logic were implemented and subjected to software-level testing. However, replay testing through the complete live MQTT message-processing path was not completed.

Limitation: The software tests provide evidence for individual security mechanisms but do not establish complete end-to-end security under all network and failure conditions.

8.4.4 Automated Load Disconnection

The MOSFET load-control interface is present, but automated load switching remains disabled in the current firmware configuration.

Limitation: Automatic load disconnection must not be claimed as a verified protection feature.

8.4.5 Anomaly-Detection Evaluation

Isolation Forest and Random Forest are part of the project's anomaly-detection approach. However, comprehensive evaluation of model accuracy, false-positive rate, false-negative rate, and generalization under different operating conditions has not been established.

Limitation: The anomaly-detection component should not be described as providing validated predictive or diagnostic performance without measured evaluation results.

8.4.6 System Integration and Safety

The prototype combines embedded sensing, local alerting, encrypted MQTT telemetry, backend processing, database storage, and a dashboard. Nevertheless, not all possible end-to-end operating scenarios, network interruptions, power failures, and recovery conditions have been validated.

Furthermore, a successful prototype test does not demonstrate compliance with battery-safety standards.

Limitation: The system requires additional integration, electrical-safety, and reliability testing before it can be considered for deployment beyond controlled experimental use.

8.4.7 Summary of Limitations

Table 8.2: Identified Limitations and Further Work

Identified limitation	Further work required
Temperature accuracy not fully calibrated	Compare readings with a calibrated reference across the intended temperature range.
INA219 measurements not validated	Validate voltage and current readings against reference instruments.
Live MQTT replay testing incomplete	Perform end-to-end tests through the actual broker and backend callback.
Automated load switching disabled	Implement and validate switching only after a safe test procedure is established.
Anomaly-detection performance not quantified	Evaluate using representative datasets and report appropriate performance metrics.
Complete integration not fully validated	Conduct systematic end-to-end and fault-recovery tests.
Safety certification not established	Perform appropriate electrical-safety assessments before any deployment claims.

8.5 Testing Summary

Testing was used to examine the major hardware and software components of Secure-BMS. The ESP32 and ADS1115 were initialized successfully, and the temperature-sensing circuit produced readings that responded to changes in sensor conditions. The comparator and buzzer demonstrated the basic operation of the local alert path.

At the software level, the project includes encrypted MQTT telemetry, backend message processing, PostgreSQL storage, and replay-protection logic. Software-level tests covered

valid message progression, duplicate and stale counters, session handling, and database transaction rollback.

The testing process also identified areas requiring further validation. These include calibrated temperature accuracy, INA219 measurement verification, live MQTT replay testing, anomaly-detection evaluation, and end-to-end system integration. Automated load switching remains disabled in the current firmware configuration.

Overall, the tests provide evidence of the prototype's implemented monitoring and software components while identifying the work needed for more comprehensive verification. The results should be interpreted within the limits of the tests actually performed, and the prototype should not be considered a certified battery safety or protection system.

9. Results and Discussion

9.1 Prototype Implementation Results

The Secure-BMS (Secure Battery Management System) prototype was developed to demonstrate battery-temperature monitoring, local alert generation, secure telemetry communication, backend processing, database storage, and dashboard-based monitoring. The implementation combines embedded hardware with software components to demonstrate the integration of sensing, communication security, and data management in a cyber-physical system.

The ESP32 NodeMCU development board serves as the primary embedded controller. The ADS1115 analog-to-digital converter is connected through the I²C interface and was detected at address `0x48`. A nominally 100 kΩ NTC thermistor with a B3950 characteristic and a 10 kΩ fixed resistor are used in the temperature-sensing circuit.

The firmware collects sensor readings using a multi-sample median-filtering approach. It also manages temperature-alert and sensor-fault states and attempts to maintain Wi-Fi connectivity. The system uses an LM339 comparator connected to GPIO26 and an active-low buzzer connected to GPIO25 for local alerting. The prototype demonstrated basic changes in sensor readings under different temperature conditions, as well as changes in comparator output and buzzer behavior during alert testing.

The software implementation includes encrypted telemetry using AES-128-GCM, MQTT communication through the `secure-bms/telemetry` topic, backend processing, PostgreSQL storage, and a Streamlit monitoring dashboard. The backend validates and decrypts incoming telemetry before processing it. Database-backed replay protection maintains session and message-counter state to identify duplicate or stale messages.

Software-level tests covered fresh messages, increasing counters, duplicate and stale counters, new sessions, previously seen inactive sessions, and transaction rollback. These tests provided evidence that the replay-protection and database logic behaved as intended in the tested cases.

The prototype implementation can therefore be summarized as follows.

Table 9.1: Prototype Implementation Results

Component	Implementation result
ESP32 controller	Firmware uploaded and executed during development.
ADS1115 ADC	Detected at I²C address `0x48`.
Temperature sensing	Sensor readings responded to changes in temperature conditions.
Local alert mechanism	Comparator state changes and buzzer operation were observed during testing.
Wi-Fi communication	Firmware includes connection and reconnection handling.
Secure telemetry	AES-128-GCM encryption and authenticated backend decryption were implemented.
MQTT communication	The telemetry pipeline uses the `secure-bms/telemetry` topic.
Backend processing	Incoming messages are decrypted, validated, and processed before storage.
Replay protection	Software-level tests covered message-counter and session-validation scenarios.
Database	PostgreSQL stores telemetry and replay-protection state transactionally.
Dashboard	A Streamlit-based monitoring interface is part of the software implementation.
Anomaly detection	Isolation Forest and Random Forest are part of the proposed detection approach; performance is not yet comprehensively validated.

The results indicate that the prototype incorporates the principal hardware and software elements required for experimental battery monitoring. However, the existence of an implemented component does not independently establish its complete end-to-end reliability.

9.2 Results Against Project Objectives

The project objectives focused on developing a battery-monitoring prototype that combines physical sensing, local alerts, secure telemetry, persistent data storage, replay protection, and centralized monitoring.

The implementation status of these objectives is summarized in Table 9.2.

Table 9.2: Evaluation Against Project Objectives

Project objective	Result and status
Monitor battery temperature	Implemented and demonstrated through the thermistor and ADC circuit. Comprehensive calibration has not been completed.
Detect temperature-alert conditions	Basic comparator and firmware alert behavior was observed during testing.
Provide local alerts	The active-low buzzer operated according to the configured alert logic.
Transmit telemetry over a	Wi-Fi and MQTT communication support are implemented;

network	comprehensive end-to-end reliability testing remains necessary.
Protect telemetry confidentiality and integrity	AES-128-GCM encryption and authenticated decryption are implemented. Broader end-to-end security validation remains incomplete.
Store telemetry persistently	PostgreSQL-based telemetry storage and transactional database operations are implemented.
Reduce duplicate and stale message acceptance	Database-backed replay protection was tested at the software and database levels. Testing through the complete live MQTT callback remains pending.
Provide centralized monitoring	A Streamlit dashboard is included for presenting available telemetry and system information.
Detect anomalous behavior	Isolation Forest and Random Forest form part of the anomaly-detection approach, but comprehensive model-performance results are unavailable.
Disconnect the load automatically under unsafe conditions	Not demonstrated as an active feature. The current firmware configuration keeps `LOAD_ENABLE = false`.

The results show that the project addresses its principal monitoring and secure-communication objectives at the prototype level. Temperature sensing and local alerting were demonstrated, while encryption, message validation, persistent storage, and replay-protection mechanisms were implemented and subjected to software-level testing.

Some objectives require further validation. In particular, temperature accuracy needs reference-based calibration, the INA219 current measurements have not been fully validated, and the complete live MQTT replay-protection path requires integration testing. Automated load disconnection is not an enabled feature of the current firmware.

Consequently, the project objectives should be regarded as achieved to different degrees rather than uniformly completed. The implementation demonstrates the feasibility of integrating battery monitoring with secure telemetry and database-backed message validation, while identifying the additional work needed for a more thoroughly verified system.

9.3 Discussion

The Secure-BMS prototype demonstrates the integration of physical sensing, embedded processing, local alerting, network communication, and software-based security mechanisms. Its main contribution is the combination of battery-monitoring functions with authenticated telemetry processing and replay protection within a single experimental cyber-physical system.

Temperature Monitoring and Local Alerting

The thermistor-based sensing circuit provides a method of estimating battery temperature from the measured divider voltage. The use of multiple samples and median filtering helps reduce the influence of isolated measurement variations. The comparator and buzzer provide a local alert path that does not depend entirely on the remote dashboard.

However, the observed response to temperature changes is not equivalent to a formal measurement-accuracy assessment. Calibration, repeatability testing, and evaluation across the intended operating range are necessary before the readings can be treated as quantitatively reliable.

Secure Telemetry and Replay Protection

The use of AES-128-GCM provides authenticated encryption for telemetry payloads. This mechanism is intended to protect message confidentiality and detect unauthorized changes to encrypted data. The MQTT broker provides message transport, while authentication and payload validation are performed by the application software.

The backend also maintains replay-protection state using message sessions and counters stored in PostgreSQL. The software-level tests indicate that the tested duplicate, stale-counter, session, and transaction scenarios were handled according to the implemented rules.

Nevertheless, the scope of these findings is limited to the tested software paths. Live end-to-end replay testing through the complete MQTT callback was not completed. Additional testing is required to evaluate the behavior of the integrated system during reconnection, repeated delivery, process restarts, and other network disruptions.

Database and Dashboard Integration

PostgreSQL provides persistent storage for telemetry and replay-protection state. Transactional handling helps preserve consistency when related database operations succeed or fail. The Streamlit dashboard provides a user-facing interface for inspecting the information made available by the software pipeline.

The usefulness of the dashboard depends on the completeness and validity of the incoming data. Missing measurements, invalid sensor states, communication interruptions, or incomplete integration may affect the information available for display. These scenarios should be included in future integration tests.

Anomaly Detection

Isolation Forest and Random Forest are considered in the anomaly-detection component. These methods can support the identification or classification of unusual telemetry patterns when supplied with suitable data and appropriate model configurations.

However, no comprehensive model-performance evaluation has been established for the current prototype. Therefore, the anomaly-detection component should not be described as having demonstrated a particular accuracy, false-alarm rate, or predictive capability without supporting experimental results.

Overall, the discussion indicates that Secure-BMS provides a useful experimental foundation for combining physical monitoring and software-based security. Its results demonstrate implemented mechanisms and selected successful tests, but further validation is required to establish comprehensive reliability and performance.

9.4 Limitations

Although the prototype demonstrates several important functions, a number of limitations remain.

1. Temperature calibration: The thermistor readings were observed to respond to changes in temperature, but a comprehensive comparison against a calibrated reference instrument was not completed. Absolute measurement accuracy has therefore not been established.

2. Current measurement: The INA219 was detected at I²C address `0x40`, but its voltage and current measurements were not fully validated. Verified current and power monitoring cannot be claimed on the basis of device detection alone.

3. Automated load control: The MOSFET interface is present, but the firmware keeps `LOAD_ENABLE = false`. Consequently, automatic load disconnection has not been demonstrated as a functioning protection mechanism.

4. End-to-end security testing: Encryption, authenticated decryption, and replay-protection logic were implemented, and selected software-level tests were performed. However, live replay testing through the complete MQTT callback and broader end-to-end security testing remain incomplete.

5. Anomaly-detection performance: Comprehensive evaluation of Isolation Forest and Random Forest using representative datasets has not been established. Quantitative model-performance claims would require documented experiments and suitable evaluation metrics.

6. System reliability: The prototype has not undergone comprehensive testing across all operating conditions, including extended operation, repeated network interruptions, power failures, and recovery scenarios.

7. Battery safety: The prototype has not been established as a certified battery protection system. It should not be relied upon as the sole safety mechanism for a lithium-ion battery. Additional electrical-safety assessment and controlled validation would be required before considering deployment beyond experimental use.

These limitations define the current scope of the project. They also identify the most important areas for further development and validation.

9.5 Overall Evaluation

The Secure-BMS project demonstrates a prototype architecture for monitoring battery temperature and combining local alerting with secure network telemetry, backend processing, persistent storage, and centralized visualization.

The hardware tests demonstrated basic operation of the temperature-sensing circuit, comparator, and buzzer. The software implementation includes AES-128-GCM authenticated encryption, MQTT-based telemetry transport, backend validation, PostgreSQL storage, and database-backed replay protection. Software-level tests provided evidence of correct handling for selected message-counter, session-validation, and transaction scenarios.

At the same time, the evaluation identified areas that remain incomplete. Temperature measurement accuracy requires calibration, current sensing requires further validation, end-to-end replay testing is pending, and anomaly-detection performance has not been comprehensively measured. Automated load switching remains disabled, and the system has not been certified for battery safety.

Based on these findings, Secure-BMS can be evaluated as an experimental cyber-physical monitoring prototype that demonstrates the integration of sensing, alerting, authenticated telemetry, and replay-protection mechanisms. The results support the feasibility of the design within the tested conditions, while further testing is needed before making broader claims about reliability, predictive performance, or safety.

The project provides a foundation for future improvements, including calibrated sensing, validated current measurement, end-to-end security testing, systematic anomaly-detection evaluation, and carefully controlled load-management experiments. These improvements would strengthen the evidence supporting the system's functionality and help establish a clearer basis for evaluating its performance under realistic operating conditions.

10. Conclusion and Future Enhancements

10.1 Conclusion

The Secure-BMS (Secure Battery Management System) project presents an experimental cyber-physical system for monitoring a physical 18650 lithium-ion battery cell and integrating embedded sensing with secure telemetry communication, backend processing, persistent data storage, and dashboard-based visualization.

The hardware prototype uses an ESP32 NodeMCU development board as its primary controller. Temperature measurements are obtained using a nominally 100 kΩ NTC thermistor with a B3950 characteristic and an ADS1115 analog-to-digital converter. The embedded firmware applies multi-sample median filtering, maintains temperature-alert and sensor-fault states, and supports Wi-Fi connectivity. A comparator and active-low buzzer provide a local alert mechanism when the configured alert condition is reached.

A key aspect of Secure-BMS is the integration of security mechanisms into the telemetry pipeline. AES-128-GCM authenticated encryption is used to protect telemetry payloads, while MQTT provides message transport between the embedded device and the backend. The backend authenticates and decrypts incoming messages before processing them. Database-backed replay protection uses session information and message counters to identify duplicate or stale messages. PostgreSQL provides persistent telemetry storage and transactional handling of related replay-protection state.

The software implementation also includes a FastAPI backend and a Streamlit dashboard for accessing and presenting available system information. Isolation Forest and Random Forest form part of the project's anomaly-detection approach, although their performance has not yet been comprehensively validated.

The tests conducted during development demonstrated basic temperature-sensing behavior, comparator-state changes, buzzer operation, and selected software-level replay-protection and database transaction scenarios. These results provide evidence that important hardware and software components have been implemented and tested under the conditions examined.

However, several limitations remain. Temperature measurements require comprehensive calibration, INA219 current measurements have not been fully validated, and end-to-end replay testing through the live MQTT callback remains incomplete. Automated load switching is disabled in the current firmware configuration. In addition, the prototype has not undergone comprehensive battery-safety certification or full system reliability validation.

In conclusion, Secure-BMS establishes an experimental foundation for combining battery monitoring with authenticated telemetry, replay protection, and centralized data management. The project demonstrates the feasibility of integrating these capabilities within a cyber-physical prototype while identifying the additional verification required to improve its reliability and practical applicability. The prototype should be considered a monitoring and research testbed rather than a certified battery protection system.

10.2 Future Enhancements

Several enhancements can be introduced to improve the accuracy, reliability, security, and usability of Secure-BMS.

10.2.1 Improved Temperature Measurement

The temperature-sensing circuit can be calibrated against a suitable reference thermometer over the intended operating range. Calibration can help quantify measurement error and improve the reliability of temperature estimates.

Further improvements may include evaluating sensor placement, thermal contact, sampling intervals, filtering parameters, and temperature-response time. These changes would help establish more reliable measurement behavior under controlled conditions.

10.2.2 Validated Voltage and Current Monitoring

The current prototype includes an INA219 sensor interface, but its measurements have not been fully validated. Future work should verify the sensor connections and measurement configuration, then compare its voltage and current readings with suitable reference instruments.

Once validated, voltage and current measurements could be integrated with temperature readings to provide a more comprehensive view of battery behavior. Derived quantities such as power and accumulated energy could then be calculated where the measurement data and sampling method support them.

10.2.3 Automated Load Management

The hardware includes an IRLZ44N MOSFET interface for load control, but automated switching is disabled in the current firmware. Future development could introduce a carefully

validated load-control state machine with defined operating thresholds, fault handling, and recovery behavior.

Before enabling automated switching, the design should be evaluated using a controlled test setup and an appropriate load. The implementation should also account for microcontroller resets, sensor failures, unexpected switching states, and loss of communication. Independent hardware protection should be considered wherever required by the electrical design.

10.2.4 Enhanced Telemetry Security

The existing AES-128-GCM mechanism and database-backed replay protection provide a basis for further security evaluation. Future work could include systematic end-to-end testing through the live MQTT broker and backend callback.

Additional improvements may include secure key provisioning, controlled key rotation, protected credential storage, stronger session lifecycle management, and more comprehensive testing of duplicate delivery, reconnect behavior, malformed payloads, and process restarts. MQTT transport security and access controls should also be configured appropriately for the intended deployment environment.

10.2.5 Improved Anomaly Detection

The anomaly-detection component can be strengthened through systematic evaluation of Isolation Forest and Random Forest using representative telemetry datasets. The evaluation should use clearly defined training and testing data and report appropriate metrics, such as precision, recall, F1-score, and false-positive rate where applicable.

Future models could consider multiple input variables, including temperature, voltage, current, and changes in measurements over time, provided these measurements are validated and available. The system should distinguish experimental anomaly scores from confirmed battery faults and should not rely on an unvalidated model as the sole safety mechanism.

10.2.6 Dashboard Improvements

The Streamlit dashboard could be extended to present temperature trends, validated voltage and current measurements, alert history, communication status, and replay-protection events. Historical charts and filtering options could help users examine changes over time.

Additional features could include configurable alert thresholds, downloadable telemetry records, clearer sensor-fault indicators, and system-health summaries. Any dashboard controls that affect hardware behavior would require authentication, authorization, and appropriate safety validation before being enabled.

10.2.7 Comprehensive Testing and Reliability Evaluation

Future testing should cover the complete telemetry path from physical sensing through encryption, MQTT transmission, backend validation, database storage, and dashboard display.

Testing should also include Wi-Fi interruptions, broker restarts, database failures, invalid sensor readings, repeated messages, counter-boundary conditions, and recovery after power loss. The results should be documented using reproducible procedures, recorded test outcomes, and clearly defined acceptance criteria.

These enhancements would improve the evidence available to assess the prototype's accuracy, security, and reliability.

10.3 Future Scope

Secure-BMS can be extended into a more comprehensive battery-monitoring and research platform. Its current architecture provides a starting point for further work in cyber-physical system monitoring, secure communication, fault detection, and battery-data analysis.

10.3.1 Multi-Cell Battery Monitoring

The system could be extended from a single-cell testbed to a multi-cell battery arrangement. This would require additional sensing channels, cell-level measurement management, suitable isolation and protection considerations, and methods for comparing individual cell behavior.

Multi-cell monitoring could help identify differences in temperature and electrical behavior between cells. However, such an extension would require a redesigned and validated hardware architecture rather than simply connecting additional cells to the existing prototype.

10.3.2 Cloud-Based Monitoring

The local MQTT and backend architecture could be extended to a securely configured cloud environment. Remote storage and visualization could allow authorized users to monitor telemetry from multiple test devices and examine historical trends.

Such a deployment would require secure device authentication, encrypted network connections, protected key management, access control, reliable connectivity, and appropriate data-retention policies. Cloud deployment should follow successful validation of the local system.

10.3.3 Predictive Analytics and Battery Health Estimation

With sufficiently accurate and representative datasets, future versions could investigate battery-state estimation and predictive analytics. Possible research directions include state-of-charge estimation, state-of-health estimation, degradation analysis, and detection of unusual thermal or electrical behavior.

These capabilities would require additional validated measurements and suitable reference data. Model performance would need to be evaluated under representative operating conditions before predictions could be considered reliable.

10.3.4 Digital Twin Integration

A future version could incorporate a digital representation of the monitored battery and its operating conditions. Such a digital twin could combine measured telemetry with a model of expected battery behavior to support visualization, simulation, and comparison between measured and predicted values.

The usefulness of this approach would depend on the accuracy of the physical measurements and the validity of the underlying model. Model assumptions, prediction errors, and differences between simulated and physical behavior would need to be documented.

10.3.5 Advanced Fault Detection and Recovery

The system could be extended to identify a broader set of fault conditions, including sensor disconnection, abnormal temperature trends, communication loss, inconsistent measurements, and unexpected system restarts.

A future fault-management framework could associate each detected condition with a defined response, event record, and recovery procedure. Any response involving battery isolation or load switching would require dedicated hardware design and safety validation.

10.3.6 Scalable Monitoring and Research Applications

The architecture could eventually support multiple battery-monitoring nodes reporting to a centralized platform. This would allow researchers to compare telemetry from different test conditions, examine patterns across datasets, and evaluate detection algorithms using recorded experimental data.

Potential applications include academic research, laboratory battery experiments, and prototype monitoring systems for battery-powered equipment. Suitability for any practical deployment would depend on further validation, electrical-safety assessment, and the requirements of the intended application.

Overall, the future scope of Secure-BMS extends beyond basic temperature monitoring toward a more comprehensive platform for secure battery telemetry, data analysis, and experimental fault detection. Achieving this scope will require incremental development, reliable measurement data, reproducible testing, and careful validation of all hardware and software functions.

11. References

1. Espressif Systems. *ESP32 Series Datasheet*. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
2. Texas Instruments. *ADS111x Ultra-Small, Low-Power, I²C-Compatible, 860-SPS, 16-Bit ADCs Datasheet*. Available at: https://www.ti.com/lit/ds/symlink/ads1115.pdf
3. Texas Instruments. *LM339 Quad Differential Comparator Datasheet*. Available at: https://www.ti.com/lit/ds/symlink/lm339.pdf
4. Infineon Technologies. *IRLZ44N HEXFET Power MOSFET Datasheet*. Available at: https://www.infineon.com/
5. Eclipse Foundation. *Eclipse Mosquitto: An Open-Source MQTT Broker*. Available at: https://mosquitto.org/

6. OASIS. *MQTT Version 3.1.1 Specification*. Available at: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html
7. National Institute of Standards and Technology (NIST). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. Available at: https://csrc.nist.gov/pubs/sp/800/38/d/final
8. PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available at: https://www.postgresql.org/docs/
9. FastAPI. *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/
10. Streamlit. *Streamlit Documentation*. Available at: https://docs.streamlit.io/
11. Scikit-learn Developers. *Scikit-learn User Guide: Ensemble Methods and Isolation Forest*. Available at: https://scikit-learn.org/stable/user_guide.html
12. Python Software Foundation. *Python Documentation*. Available at: https://docs.python.org/3/
13. Eclipse Paho Project. *Eclipse Paho MQTT Python Client Documentation*. Available at: https://eclipse.dev/paho/
14. pytest Development Team. *pytest Documentation*. Available at: https://docs.pytest.org/
15. Arduino. *Arduino Documentation*. Available at: https://docs.arduino.cc/

Appendix A: GitHub Repository and Demonstration Video

A.1 GitHub Repository

The GitHub repository will serve as the central location for maintaining and documenting the Secure-BMS source code. It can contain the ESP32 firmware, backend services, database scripts, anomaly-detection modules, dashboard code, and project documentation, depending on the components implemented.

Repository details

- Project title: Secure-BMS
- Repository URL: https://github.com/SiddharthPNair/Secure-BMS
- Programming languages: C/C++ and Python, as applicable
- Version control: Git and GitHub

The repository is intended to support source-code maintenance, version tracking, reproducibility, and future development.

A.2 Demonstration Video

The demonstration video provides a visual overview of the Secure-BMS prototype and its implemented functions. It can be used to demonstrate the physical hardware, temperature-sensing circuit, local alert mechanism, and available software components.

The demonstration may include the ESP32 development board, thermistor-based temperature sensing, comparator output, buzzer operation, MQTT telemetry communication, backend

processing, database storage, and dashboard visualization, provided each feature is shown working in the recorded demonstration.

Demonstration Video Link:
https://drive.google.com/drive/folders/14AZvPNqMxzKMOkfxqLJSX7SbiOEN1cnO?usp=sharing

The video should present the prototype under controlled testing conditions and distinguish between demonstrated functionality and features that remain incomplete or unvalidated.